

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

09.03.01 Информатика и вычислительная техника
(направление подготовки/специальность)
Программное обеспечение мобильных систем
(профиль/специализация)
Очная
(форма обучения)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

(вид практики)

Тип практики Технологическая (проектно-технологическая) практика
на предприятии ООО «Бюро 1440»
(наименование профильной организации/структурного подразделения СибГУТИ)

ТЕМА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Coming soon

Выполнил:

студент института информатики и вычислительной техники Любимов Кирилл Алексеевич
группа ИА-331

_____/_____
«__» _____ 202__ г.

(подпись)

(ФИО)

Проверил¹

Руководитель практики от профильной
организации

_____/ Андреев А. В. /
(подпись) (ФИО)

«__» _____ 202__ г.

Проверил:

Руководитель практики от СибГУТИ

_____/ Брагин К. И. /
(подпись) (ФИО)

«__» _____ 202__ г.

отметка ² _____ «__» _____ 202__ г.

Новосибирск 2025

¹ В случае прохождения практики в профильной организации

² Заполняется во время промежуточной аттестации

План-график проведения
Производственной практики
вид практики
Любимов Кирилл Алексеевич
Фамилия Имя Отчество студента

института ИВТ, курса 3, гр. ИА-331

Направление: 09.03.01 Информатика и вычислительная техника

Код – Наименование направления (специальности)

Направленность (профиль)/ специализация: Программное обеспечение мобильных систем

Место прохождения практики: г. Новосибирск, ул. Бориса Богаткова, д. 51, ауд. 469

Объем практики: 360/10 часов/3Е

Тип практики: Технологическая (проектно-технологическая) практика

Срок практики: с 16.09.2025 по 26.05.2026 (раз в неделю)

Содержание практики³:

Тема индивидуального задания практики Coming soon

Наименование видов деятельности	Дата (начало – окончание)
Архитектура Adalm Pluto SDR. GNU Radio. Построение радио-приёмника	16 сентября, 2025
Знакомство с библиотеками Soapy SDR, Libiio для работы с Adalm Pluto SDR. Инициализация SDR-устройства. Работа с буфером: получение цифровых IQ-отсчетов	23 сентября, 2025
Работа с библиотеками Soapy SDR, Libiio. Формирование и передача с SDR сигналов произвольной формы. Примеры формирования I/Q-сэмплов произвольной формы. Работа с буфером приема SDR	30 сентября, 2025, 7 октября, 2025
Имитация аналоговой передачи звука и его прием с использованием SDR. Анализ влияния чувствительности приемника и усиления передатчика на качество принятых отсчетов сигнала (сэмплов)	14 октября, 2025, 21 октября, 2025
Дискретная свертка. Реализация приема и передачи BPSK-сигналов	28 октября, 2025, 11 ноября, 2025
Прием QPSK BPSK, прием на согласованный фильтр, глазковая диаграмма, поиск оптимального отсчетного значения и необходимость символьной синхронизации	18 ноября, 2025
Программная реализация детектора временной ошибки (синхронизация приемника и передатчика) на SDR. Написание функций петли (контура) синхронизации	25 ноября, 2025 2 декабря, 2025
	, 2025
	, 2025
	, 2025
	, 2025

В соответствии с рабочей программой практики

Руководитель практики от профильной

³ В случае прохождения практики в профильной организации

« ____ » _____ 202_ г.

_____/ Андреев А. В. /
(подпись) (ФИО)

« ____ » _____ 202_ г.

_____/ Брагин К. И. /
(подпись) (ФИО)

(ФИО студента)

Уровень компетенций: высокий, средний, низкий, не сформирована

Руководитель практики от СибГУТИ:

Старший преподаватель
Кафедры ТС и ВС

должность руководителя практики _____
подпись

Брагин К.И.

ФИО руководителя практики

« __ » июля 202_ г.

СОДЕРЖАНИЕ

1	ПРАКТИКА: АРХИТЕКТУРА SDR СИСТЕМЫ	3
2	ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LİBİPO	11
3	РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LİBİPO.	20
4	ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С ИСПОЛЬЗОВАНИЕМ SDR.	24
5	ДИСКРЕТНАЯ СВЕРТКА. РЕАЛИЗАЦИЯ ПРИЕМА И ПЕРЕДАЧИ BPSK СИГНАЛОВ....	28
6	ПРИЕМ QPSK BPSK, ПРИЕМ НА СОГЛАСОВАННЫЙ ФИЛЬТР, ГЛАЗКОВАЯ ДИАГРАММА, ПОИСК ОПТИМАЛЬНОГО ОТСЧЕТНОГО ЗНАЧЕНИЯ	35
7	ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ(СИНХРОНИЗАЦИЯ ПРИЕМНИКА И ПЕРЕДАТЧИКА) НА SDR	42

1 ПРАКТИКА

АРХИТЕКТУРА SDR СИСТЕМЫ

УСТАНОВКА ПО, НАСТРОЙКА УСТРОЙСТВА

Ссылка на GitHub

Цель практики:

Узнать, что такое SDR, изучить принципы его работы и внутреннюю архитектуру на базовом уровне. Познакомиться с инструментом GNU Radio и создать с его помощью программу для SDR, позволяющую принимать радио.

Краткие теоретические сведения

Что такое SDR?

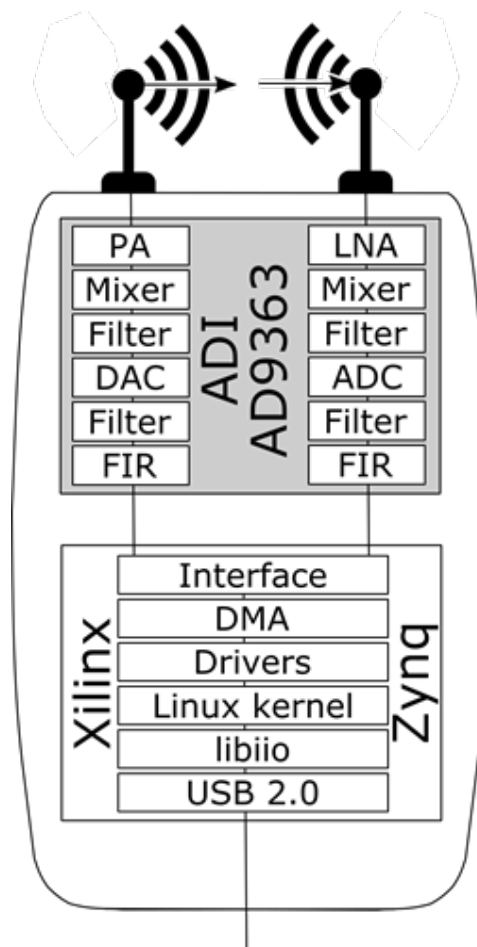


Рис. 1 — ADALM Pluto

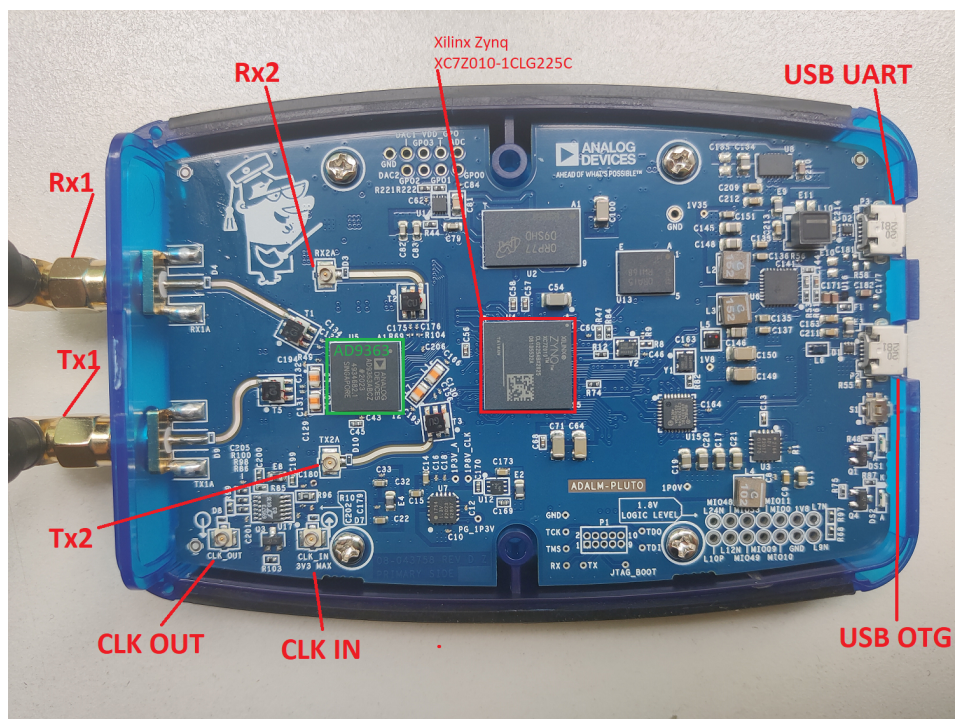


Рис. 2 — ADALM Pluto

Обучающая платформа PlutoSDR может взаимодействовать с:

1. Matlab, Simulink
2. GNU Radio
3. C, C++ при помощи дополнительных библиотек
4. C#
5. Среда языка Python

В начале данного курса мы наладим взаимодействие PlutoSDR с языком Python. Далее напишем программы под другие платформы, сравним разницу по времени обработки сигналов, простоте написания кода и редактирования.

Чип AD9363

Программируемый РЧ приемопередатчик, возможности которого позволяют использовать его для построения микро- (фемто-) сот мобильной связи 3G, 4G и 5G (в некоторых конфигурациях).

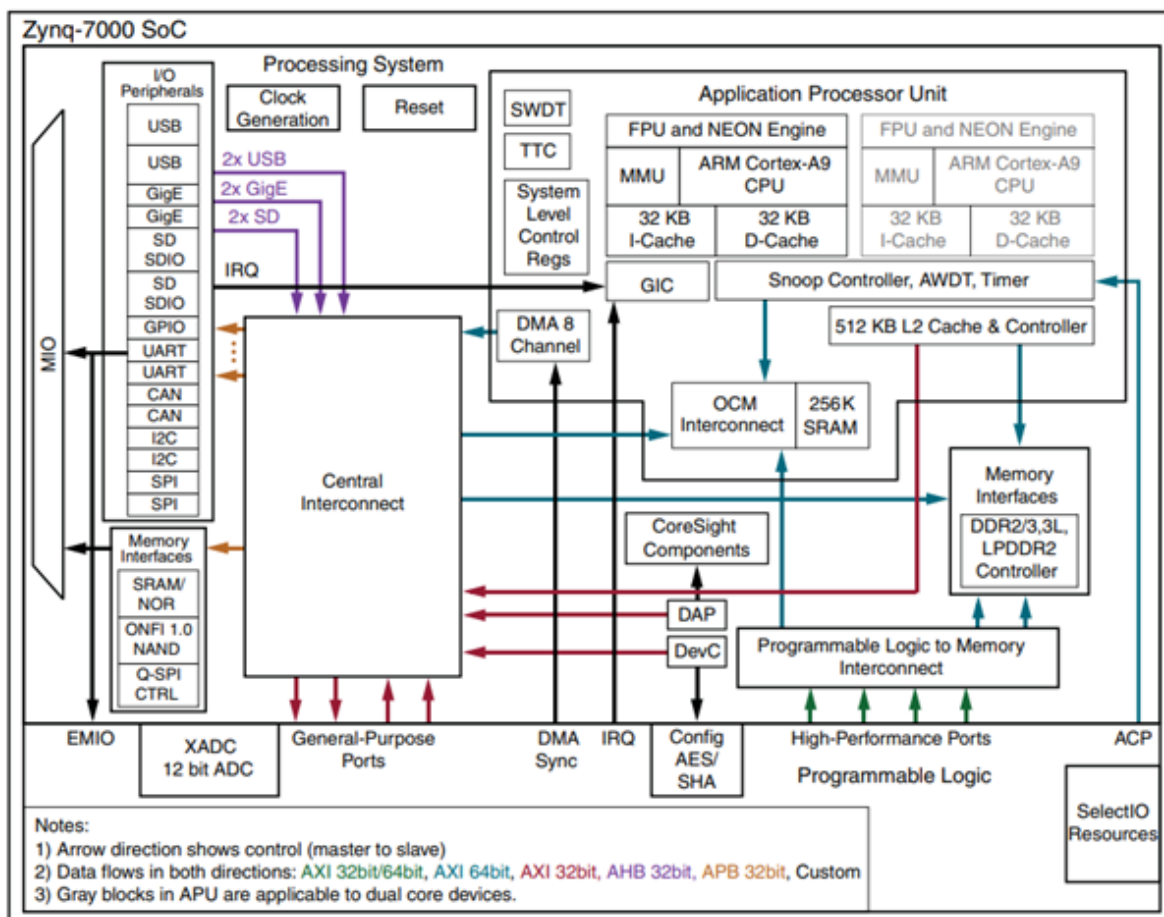


Рис. 3 — ПЛИС Xilinx Zynq

Основные характеристики

- **12-битный** ЦАП/АЦП (Цифро-аналоговый/Аналого-цифровой Преобразователь)
- Поддерживаемые несущие частоты от 90 [МГц] до **3.8** [ГГц]
- Поддерживает временной и частотный дуплексы (**TDD, FDD**)
- Ширина полосы частот: **20** [МГц]
- Шумы в приемнике: **3** [dB]
- EVM (Error Vector Magnitude): **-34** [dB]
- Tx noise: **< -157** [dBm/Hz]
- **2 Rx, 2 Tx**

Структурная схема Zynq

Каждый Zynq состоит из одного или двух ядер ARM Cortex-A9 (ARM v7), кэш L1 у каждого ядра свой, кэш L2 общий. Поддерживаемая оперативная память имеет стандарты DDR3, DDR3L, DDR2, LPDDR-2. Максимальный объем оперативной памяти равен 1 Гбайт (2 микросхемы по 4 Гбит). Максимальная тактовая частота оперативной памяти 525 МГц. Операционные системы: Standalone

(bare-metal) и Petalinux. Процессорный модуль общается с внешним миром и программируемой логикой с помощью портов, объединенных в группы:

- MIO (multiplexed I/O)
- EMIO (extended multiplexed I/O)
- GP (General-Purpose Ports)
- HP (High-Performance Ports)
- ACP (Accelerator coherency port)

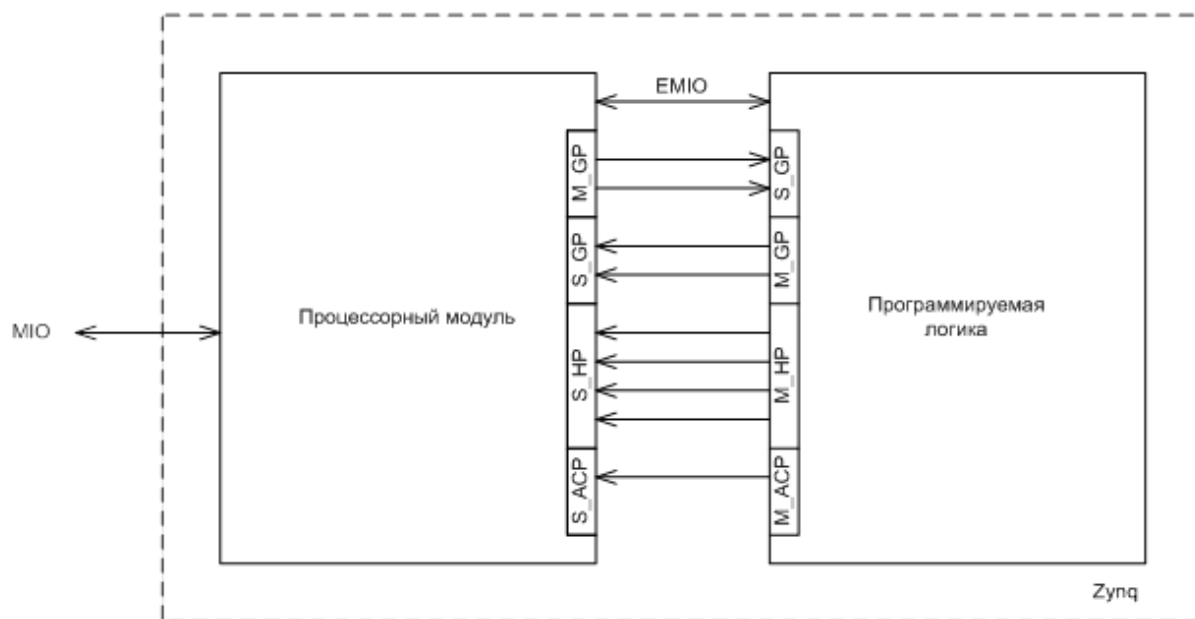


Рис. 4 — Структурная схема Zynq

Схема интерфейсов Zynq

**Буквы S и M у порта обозначают соответственно Slave и Master.*

Так как, в одном корпусе Zynq реализованы и процессорный модуль и программируемая логика, есть выводы, которые относятся к процессорному модулю и выводы, которые относятся к программируемой логике.

Порты

Порты MIO представляют собой многофункциональные порты ввода-вывода, непосредственно подключенные к выводам процессорной системы (Processing System, PS). Ключевые характеристики:

- **Количество:** 54 порта (в большинстве конфигураций)
- **Назначение:** подключение периферийных устройств PS к внешним выводам кристалла
- **Особенности:** мультиплексирование функций на одних и тех же физических выводах

MIO

Порты MIO подключены к выводам процессора. С помощью MIO могут быть подключены следующие периферийные устройства процессорного модуля:

- USB-контроллер – 2 шт
- Gigabit Ethernet контроллер – 2 шт
- SD/SDIO контроллер – 2 шт
- UART – 2 шт
- CAN – 2 шт
- I2C – 2 шт
- SPI – 2 шт
- GPIO. Все выводы можно использовать как выводы общего назначения

Так же, к MIO могут быть подключены следующие устройства памяти процессорного модуля:

- QSPI контроллер
- ONFI контроллер
- SRAM/NOR контроллер

Количество MIO портов равно 54 (за исключением некоторых микросхем в корпусе CLG225, там еще меньше). Поэтому все сразу включить не удастся. Для решения этой проблемы существует группа портов EMIO.

Выполнение

Практическая работы выполнялась в программе GNU Radio. GNU Radio - это открытая платформа для разработки программных решений в области SDR. Её ключевое преимущество — визуальное проектирование радиоэлектронных систем с помощью готовых блоков, что избавляет от необходимости программировать. Собранные проекты работают непосредственно с SDR-оборудованием, таким как Adalm-Pluto или LimeSDR.

Библиотека GNU Radio содержит обширный набор компонентов для цифровой обработки сигналов (ЦОС). Сами модули разработаны на C++ для обеспечения высокой производительности, а их соединение и управление проектом осуществляется с помощью Python. Разработку можно вести как программно, используя API, так и визуально — в графической среде GNU Radio Companion (GRC).

Создание схемы в GNU Radio

Блок Options (Параметры проекта)

Данный блок определяет основные настройки всего проекта. Наиболее важные параметры:

- **Output Language (Язык выходного кода):** определяет язык генерации исполняемой программы (Python или C++)
- **Generate Options (Опции генерации):** задаёт тип используемого интерфейса

Блок Variable (Переменная)

Служит для объявления переменных. Переменная имеет имя (ID) и значение. В проекте часто определяется:

Variable ID: samp_rate

Value: 2.4М

Блок QT GUI Range (Ползунок)

Создает регулируемый ползунок для динамического изменения параметров во время работы программы.

- **Default Value:** значение при запуске
- **Start/Stop:** диапазон значений
- **Step:** шаг изменения

Блок PlutoSDR Source (Источник)

Обеспечивает подключение к SDR-модулю ADALM-Pluto и управление его параметрами.

Основные параметры:

- **IO context URI:** IP-адрес устройства
- **Sample Rate:** частота дискретизации АЦП
- **LO Frequency:** частота гетеродина
- **Buffer size:** размер буфера

Блок Low Pass Filter (ФНЧ)

Фильтр нижних частот для ограничения полосы пропускания сигнала.

Параметры:

- **Decimation:** коэффициент уменьшения частоты дискретизации
- **Cutoff Freq:** частота среза
- **Sample Rate:** исходная частота дискретизации

Блок QT GUI Frequency Sink (Спектр)

Обеспечивает визуализацию спектра сигнала в реальном времени.

Блок QT GUI Time Sink (Осциллограф)

Отображает сигнал во временной области.

Блок WBFM Receive (FM-демодулятор)

Выполняет демодуляцию широкополосного FM-сигнала.

Блок Audio Sink (Выход на звуковую карту)

Направляет аудиопоток на звуковую карту компьютера.

Сборка системы

Все блоки соединяются в графическом редакторе GRC, формируя законченную схему FM-приёмника. Программа позволяет в реальном времени:

- Принимать радиопередачи
- Наблюдать спектр и форму сигнала
- Регулировать частоту настройки

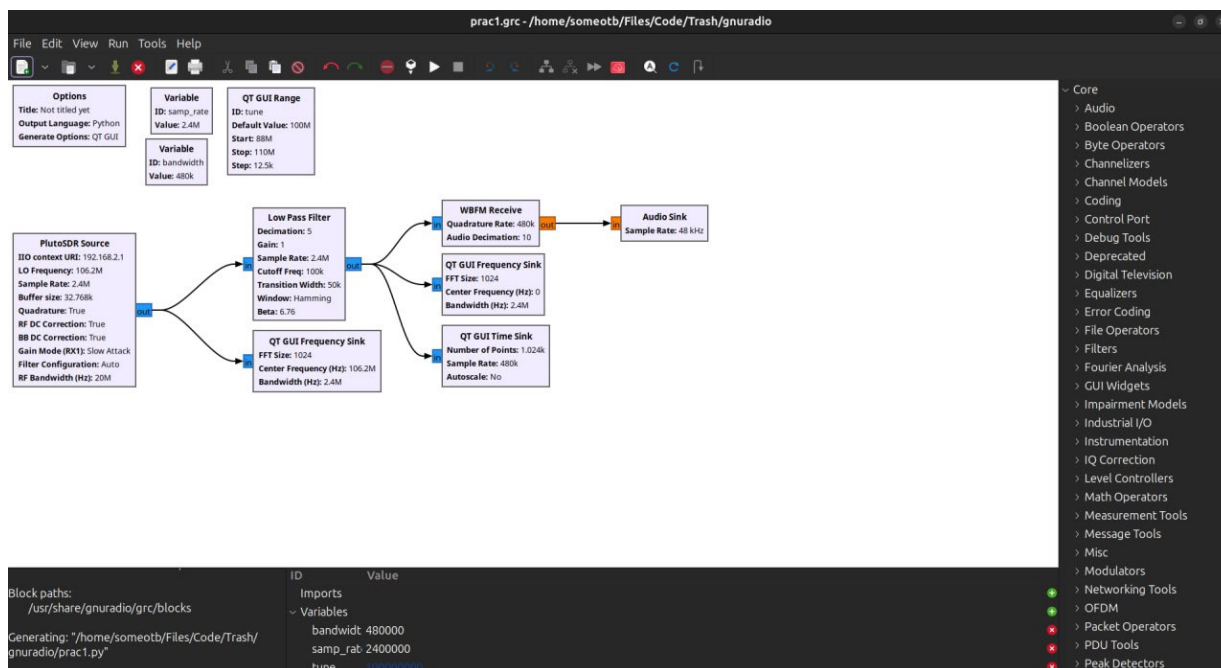


Рис. 5 — Собранная модель

Код собирается автоматически исходя из модели собранной из блоков в интерфейсе GNU Radio. И с помощью данной модели получилось поймать сигнал одной радио станции, которую можно было слушать, на других частотах звук был сильно искажен. На радио станции пока слушал, понравилась одна песня: **Девочка в платье белом - Мюзиколa**.

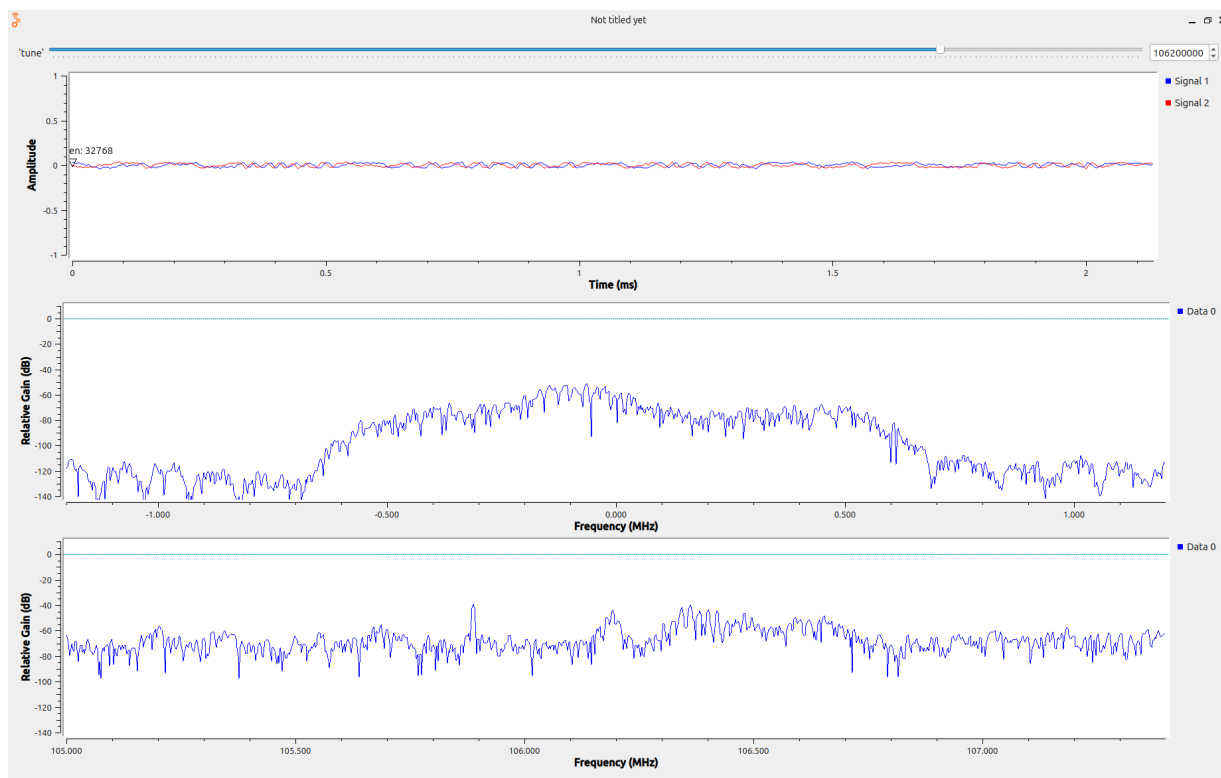


Рис. 6 — Результаты

Вывод

В результате работы были изучены принципы SDR, исследована архитектура ADALM Pluto и освоен инструмент GNU Radio. Разработана программа для приёма и воспроизведения передач в FM-диапазоне.

2 ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LIBPQ ДЛЯ РАБОТЫ С ADALM PLUTO SDR. ИНИЦИАЛИЗАЦИЯ SDR-УСТРОЙСТВА. РАБОТА С БУФЕРОМ: ПОЛУЧЕНИЕ ЦИФРОВЫХ IQ-ОТСЧЕТОВ

[Ссылка на GitHub](#)

Цель практики:

Понять как работает приёмник. Написать на C++ программу для SDR. Передать свой сигнал и принять его, чтобы проверить как всё работает.

Краткие теоретические сведения

Начнем с описания сэмпла в рамках работы с Adalm Pluto.

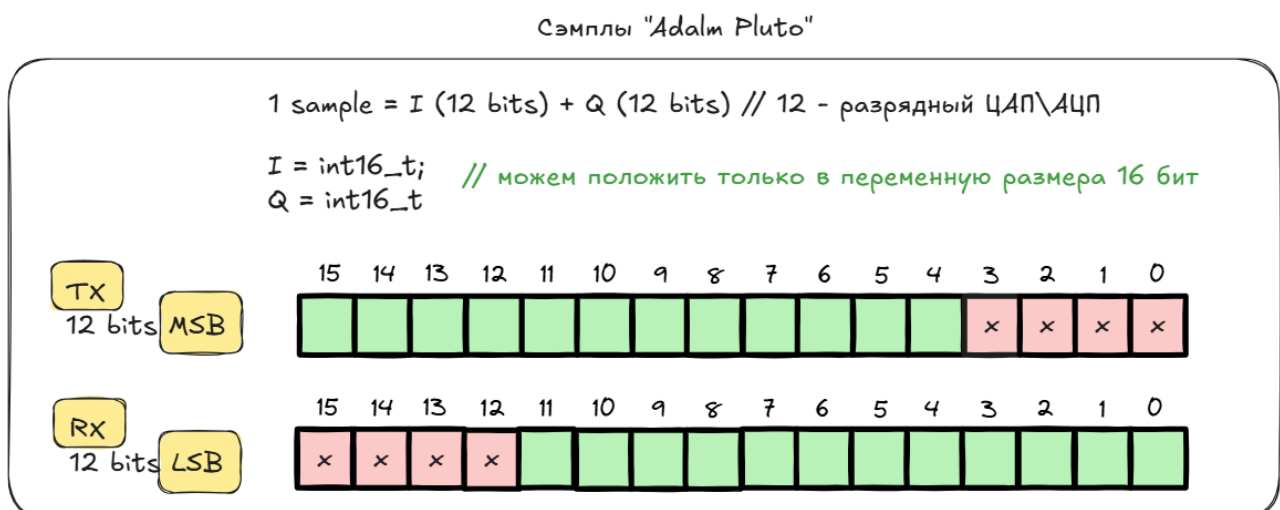


Рис. 7 — Сэмплы Adalm Pluto

Буфер, его структура, выделение памяти под буферы (для хранения сэмплов TX, RX):

Листинг 2.1 — Работа с буферами SDR

```
1 // Получение MTU
2 size_t rx_mtu = SoapySDRDevice_getStreamMTU(sdr, rxStream);
3 size_t tx_mtu = SoapySDRDevice_getStreamMTU(sdr, txStream);
4
5 // Выделение памяти под буферы
6 int16_t tx_buff[2*tx_mtu];
7 int16_t rx_buffer[2*rx_mtu];
```

Buffers structure

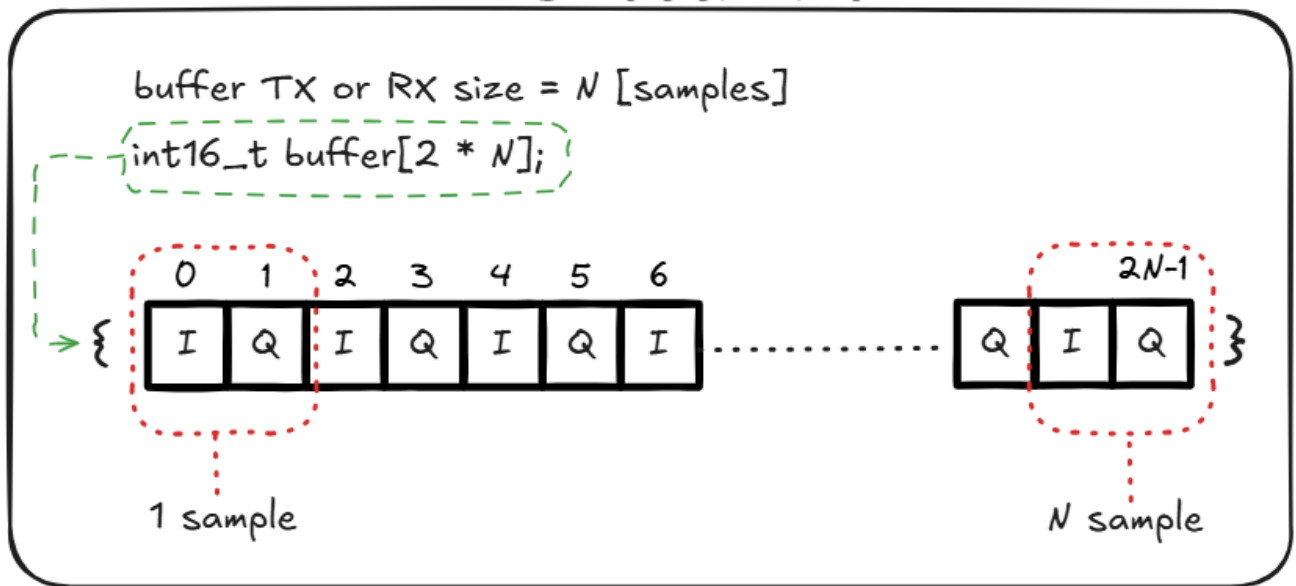


Рис. 8 — Структура буфера

Как происходит чтение сэмплов с SDR

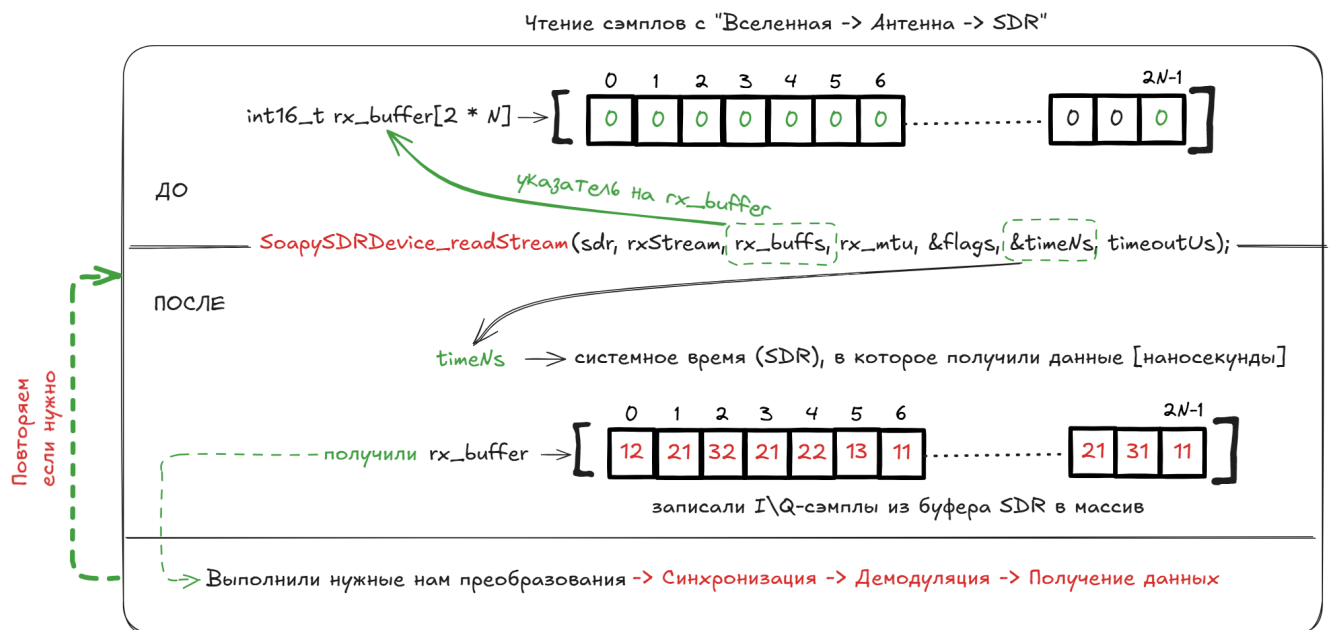


Рис. 9 — Чтение сэмплов

Листинг 2.2 — получения сэмплов с SDR

```

1 const long timeoutUs = 400000;
2 long long last_time = 0;
3 size_t iteration_count = 10;
4
5 for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
6 {
7     void *rx_buffs[] = {rx_buffer};

```

```

8  int flags;           // flags set by receive operation
9  long long timeNs;    //timestamp for receive buffer

10
11  int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
    timeNs, timeoutUs);

12
13  printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
    buffers_read, sr, flags, timeNs, timeNs - last_time);
14 }

```

Как происходит подготовка TX буфера и передача сэмплов на SDR

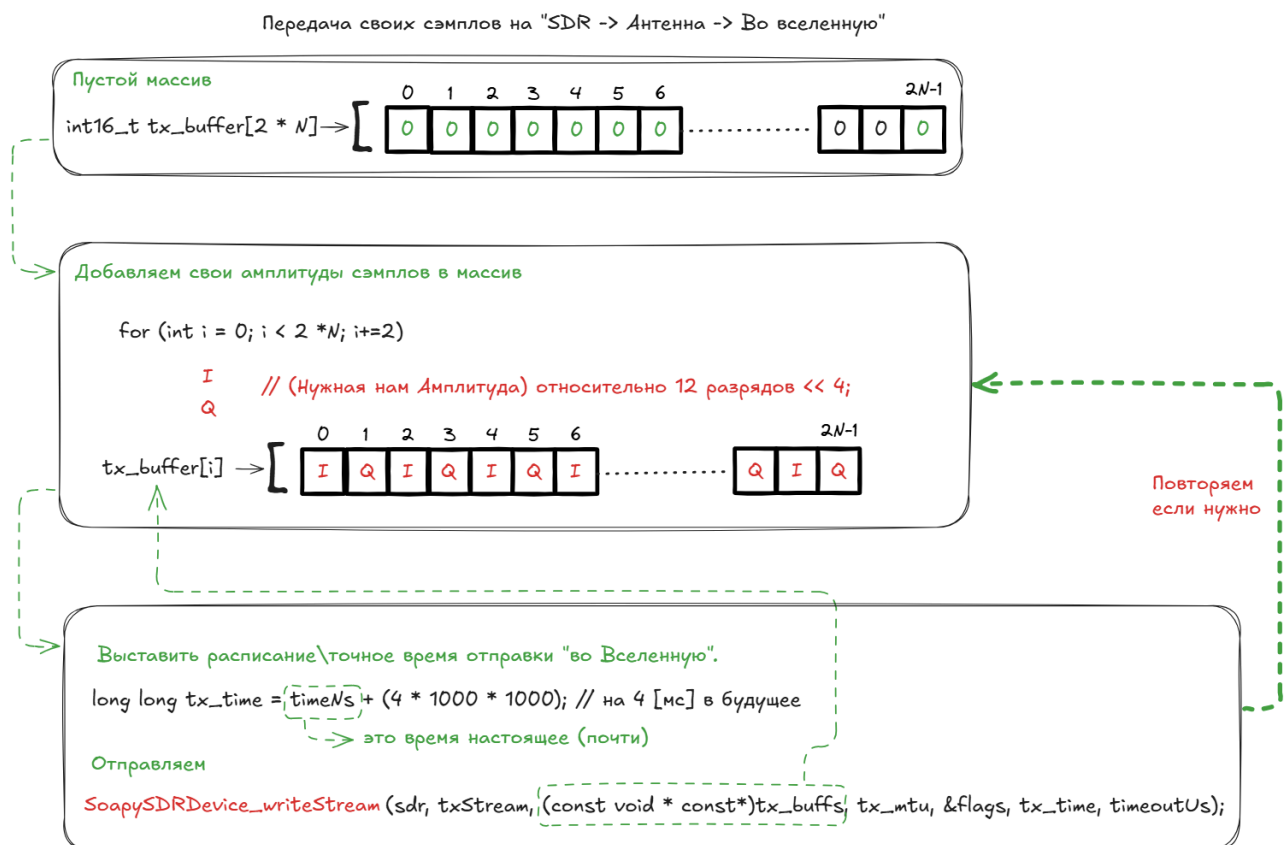


Рис. 10 — Передача сэмплов на SDR

Листинг 2.3 — Подготовка массива для передачи (TX buffer):

```

1  for (int i = 2; i < 2 * tx_mtu; i+=2)
2  {
3      tx_buff[i] = 1500 << 4; // I
4      tx_buff[i+1] = 1500 << 4; // Q
5  }
6  //prepare fixed bytes in transmit buffer
7  //we transmit a pattern of FFFF FFFF [TS_0]00 [TS_1]00 [TS_2]00 [TS_3]00 [TS_4]00
   [TS_5]00 [TS_6]00 [TS_7]00 FFFF FFFF
8  //that is a flag (FFFF FFFF) followed by the 64 bit timestamp, split into 8 bytes
   and packed into the lsb of each of the DAC words.

```

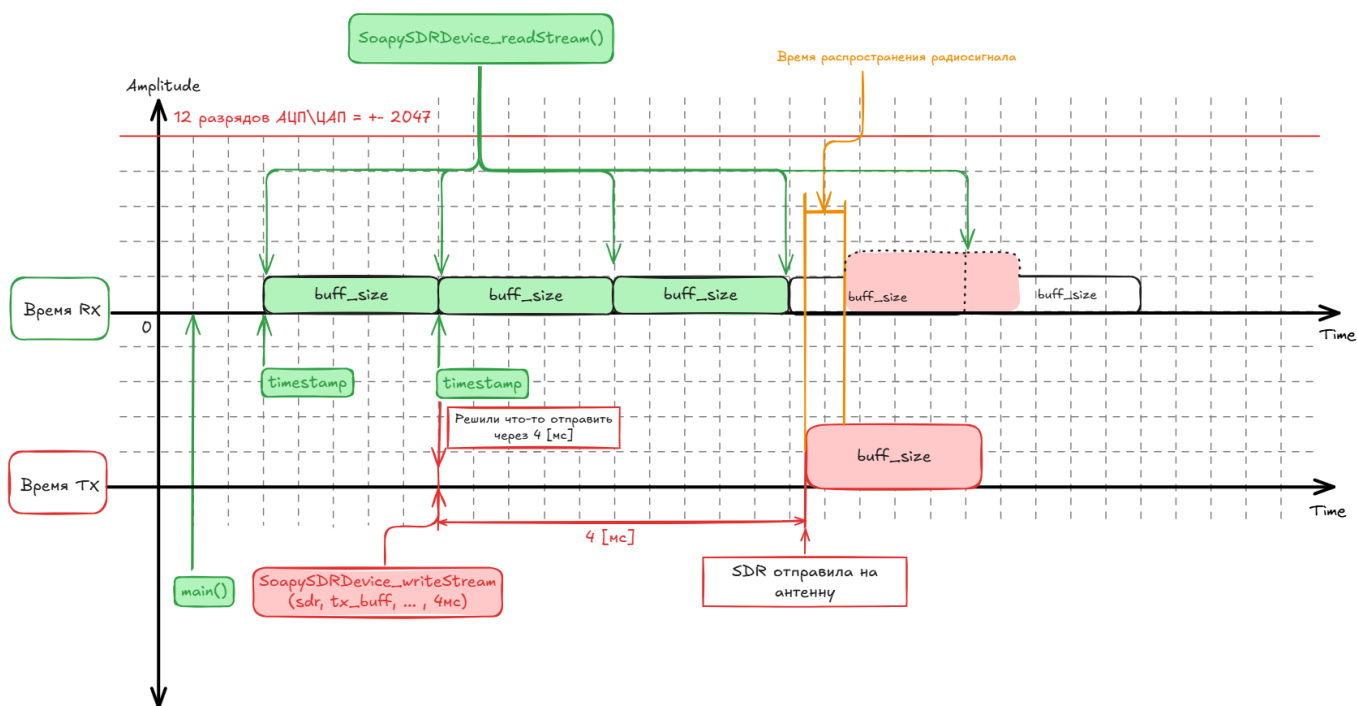


```

9 //DAC samples are left aligned 12-bits, so each byte is left shifted into place
10 for(size_t i = 0; i < 2; i++)
11 {
12     tx_buff[0 + i] = 0xffff;
13     // 8 x timestamp words
14     tx_buff[10 + i] = 0xffff;
15 }
16
17 last_time = timeNs;
18
19 long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс\[] вбудущее
20
21 for(size_t i = 0; i < 8; i++)
22 {
23     uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
24     tx_buff[2 + i] = tx_time_byte << 4;
25 }
26
27 void *tx_buffs[] = {tx_buff};
28 if( (buffers_read == 2) ){
29     printf("buffers_read: %d\\n", buffers_read);
30     flags = SOAPY_SDR_HAS_TIME;
31     int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
        tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
32     if ((size_t)st != tx_mtu)
33     {
34         printf("TX Failed: %i\\n", st);
35     }
36 }

```

Одновременный прием и передача. Ниже представлена схема работы буферов RX и TX:



Код, отражающий работу схемы:

```

1 const long timeoutUs = 400000;
2 long long last_time = 0;
3 size_t iteration_count = 10;
4
5 for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
6 {
7     void *rx_buffs[] = {rx_buffer};
8     int flags; // flags set by receive operation
9     long long timeNs; //timestamp for receive buffer
10
11     int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
        timeNs, timeoutUs);
12
13     printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
        buffers_read, sr, flags, timeNs, timeNs - last_time);
14     last_time = timeNs;
15
16     long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс[] вбудущее
17
18     for(size_t i = 0; i < 8; i++)
19     {
20         uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
21         tx_buff[2 + i] = tx_time_byte << 4;
22     }
23

```

```

24 void *tx_buffs[] = {tx_buff};
25 if( (buffers_read == 2) ){
26     printf("buffers_read: %d\n", buffers_read);
27     flags = SOAPY_SDR_HAS_TIME;
28     int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
        tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
29     if ((size_t)st != tx_mtu)
30     {
31         printf("TX Failed: %i\n", st);
32     }
33 }
34
35 }

```

Выполнение

Сперва необходимо установить все необходимые библиотеки

Листинг 2.5 — Установка SoapySDR

```

1 sudo apt-get install python3-pip python3-setuptools
2 sudo apt-get install cmake g++ libpython3-dev python3-numpy swig python3-matplotlib
3
4 git clone --branch soapy-sdr-0.8.1 https://github.com/TelecomDep/SoapySDR.git
5
6 cd SoapySDR
7 mkdir build && cd build
8
9 cmake ../
10
11 make -j`nproc` # nproc - количество потоков , например make -j16
12 sudo make install
13 sudo ldconfig

```

Листинг 2.6 — Установка Libiio

```

1 sudo apt-get install libxml2 libxml2-dev bison flex libcdk5-dev cmake
2 sudo apt-get install libusb-1.0-0-dev libaio-dev pkg-config
3 sudo apt install libavahi-common-dev libavahi-client-dev
4
5 git clone --branch v0.24 https://github.com/TelecomDep/libiio.git
6
7 cd libiio
8 mkdir build && cd build
9 cmake ../
10 make -j`nproc` # nproc - количество потоков , например make -j16
11 sudo make install

```

Листинг 2.7 — Установка LibAD9361

```
1 git clone --branch v0.3 https://github.com/TelecomDep/libad9361-iiio.git
2 cd libad9361-iiio
3
4 mkdir build && cd build
5
6 cmake ../
7
8 make -j\`nproc\` # nproc - количество потоков , например make -j16
9 sudo make install
10 sudo ldconfig
```

Листинг 2.8 — Установка SoapyPlutoSDR

```
1 git clone --branch sdr_gadget_timestamping https://github.com/TelecomDep/
  SoapyPlutoSDR.git
2 cd SoapyPlutoSDR
3
4 mkdir build && cd build
5
6 cmake ../
7
8 make -j\`nproc\` # nproc - количество потоков , например make -j16
9 sudo make install
10 sudo ldconfig
```

Соответственно подключить их в самом коде

Листинг 2.9 — Подключение библиотек

```
1 #include <SoapySDR/Device.h> // Инициализация устройства
2 #include <SoapySDR/Formats.h> // Типы данных , используемых для записи сэмплов
3 #include <stdio.h> // printf
4 #include <stdlib.h> // free
5 #include <stdint.h>
6 #include <complex.h>
```

Листинг 2.10 — Аргументы (в рамках библиотеки SoapySDR имеют формат ключ:значение)

```
1 SoapySDRKwargs args = {};
2 SoapySDRKwargs_set(&args, "driver", "plutosdr");
3 if (1) {
4     SoapySDRKwargs_set(&args, "uri", "usb:");
5 } else {
6     SoapySDRKwargs_set(&args, "uri", "ip:192.168.2.1");
7 }
8 SoapySDRKwargs_set(&args, "direct", "1");
9 SoapySDRKwargs_set(&args, "timestamp_every", "1920");
10 SoapySDRKwargs_set(&args, "loopback", "0");
11 SoapySDRDevice *sdr = SoapySDRDevice_make(&args);
12 SoapySDRKwargs_clear(&args);
```

Мы подключаем SDR к нашему ПК или ноутбуку по USB2.0, в момент запуска программы SDR начинает выполнять запрограммированные действия, в нашем случае мы отправляем десять буферов, которые записываются в бинарный файл с расширением .pcm(Pulse-Code Modulation - импульсно-кодовая модуляция), который мы далее визуализируем с помощью python, и его библиотек matplotlib, numpy

Листинг 2.11 — Код для визуализации полученных данных с SDR

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 rx = np.fromfile("/home/plutoSDR/sdr/pluto/dev/rx.pcm", dtype=np.int16)
5
6 samples = []
7 for x in range(0, len(rx), 2):
8     samples.append(rx[x] + 1j * rx[x+1])
9
10 samples = np.array(samples)
11
12 ampl = np.abs(samples)
13 phase = np.angle(samples)
14 time = np.arange(len(samples))
15
16 plt.figure(figsize=(10, 8))
17
18 plt.subplot(3, 1, 1)
19 plt.plot(time, ampl)
20 plt.title("Amplitude")
21 plt.ylabel("Amplitude")
22 plt.grid(True)
23
24 plt.subplot(3, 1, 2)
25 plt.plot(time, phase)
26 plt.title("Phase")
27 plt.ylabel("Phase (rad)")
28 plt.grid(True)
29
30 plt.subplot(3, 1, 3)
31 plt.plot(time, np.real(samples), label='I (Real)')
32 plt.plot(time, np.imag(samples), label='Q (Imag)')
33 plt.title("I and Q components")
34 plt.ylabel("Value")
35 plt.xlabel("Time (samples)")
36 plt.legend()
37 plt.grid(True)
38
39 plt.tight_layout()
40 plt.show()
```

Полученные Результаты приведены на графике, через subplot, на первом subplot показана амплитуда полученного сигнала, на втором subplot показана фаза полученного сигнала, на третьем I - реальная и Q - мнимая части.

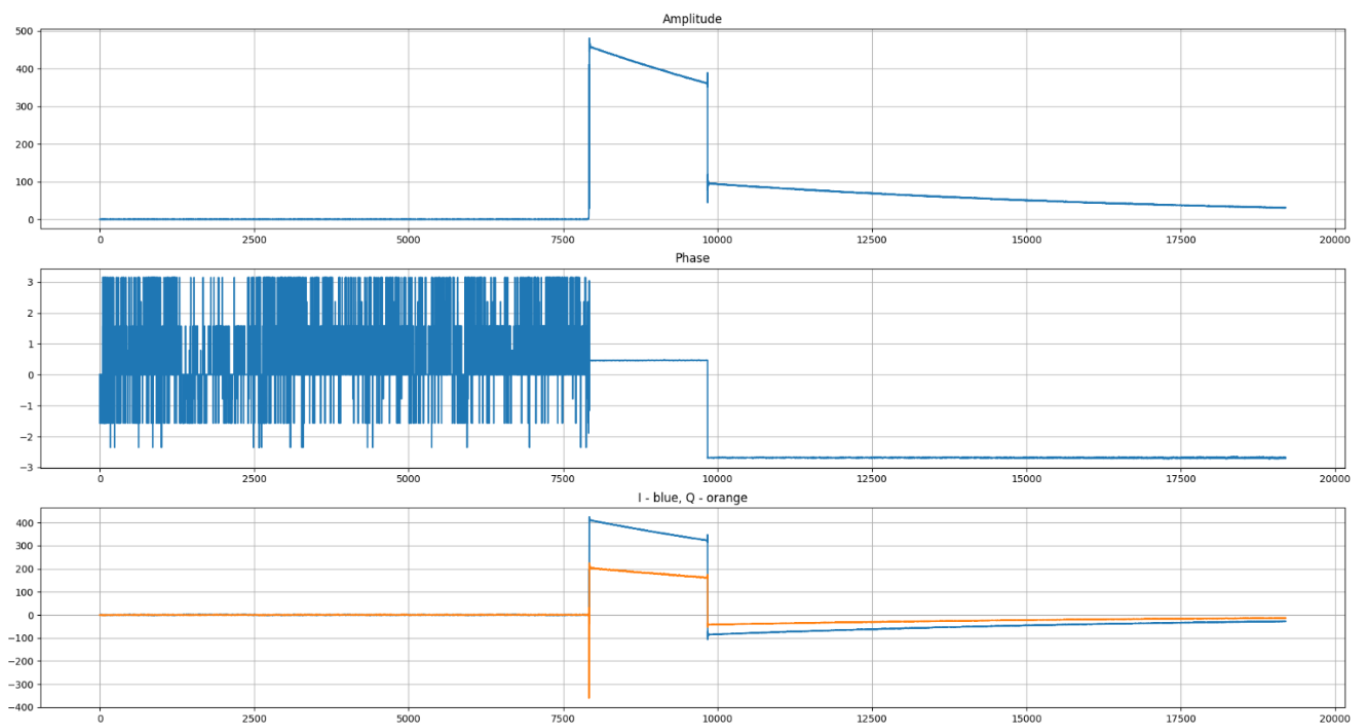


Рис. 12 — Результаты, которые мы получили с SDR

Вывод

В ходе работы я познакомился с минимальной схемой работы Pluto SDR, установил и подключил нужные библиотеки (LibAD9361, Libiio, SoapySDR, SoapyPlutoSDR), отправил сэмплы и получил их, вывел характеристики полученного сигнала на график.

3 РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LİBİO. ФОРМИРОВАНИЕ И ПЕРЕДАЧА С SDR СИГНАЛОВ ПРОИЗВОЛЬНОЙ ФОРМЫ

Ссылка на GitHub

Цель практики:

Лучше разобраться с библиотеками, сформировать сэмплы и отправить их, визуализировать полученные данные.

Краткие теоретические сведения

Вспомним структуру отправляемого буфера

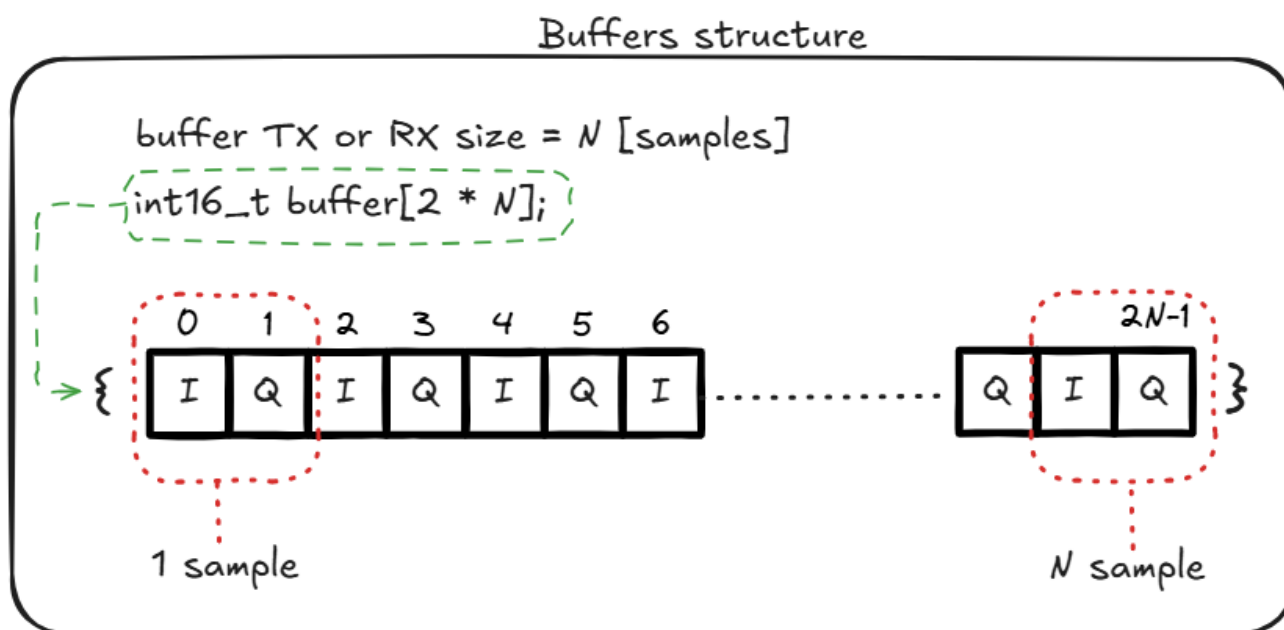


Рис. 13 — Структура буфера

Структура сэмплов

Сэмплы "Adalm Pluto"

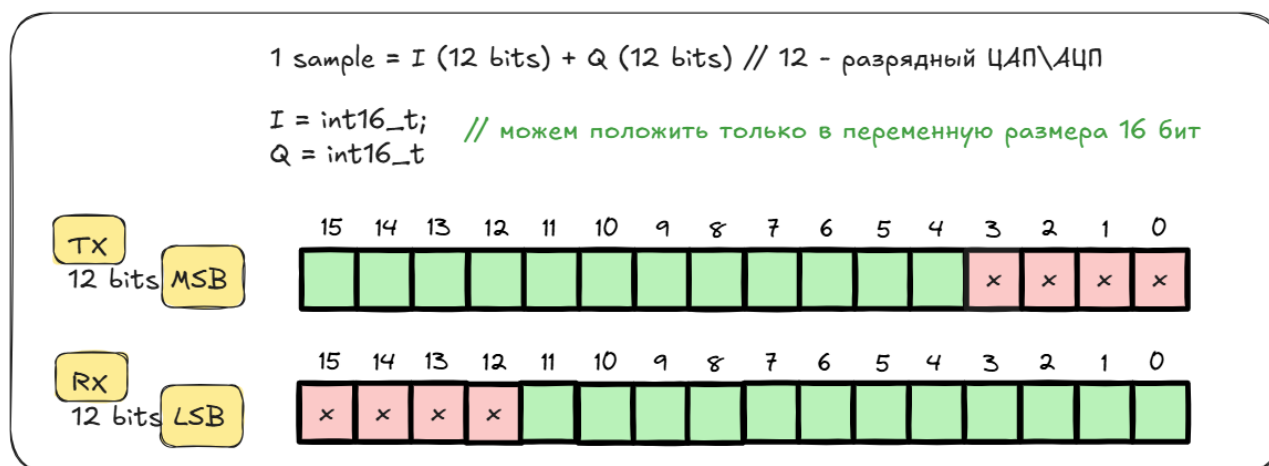


Рис. 14 — Структура сэмплов

Как читаются сэмплы

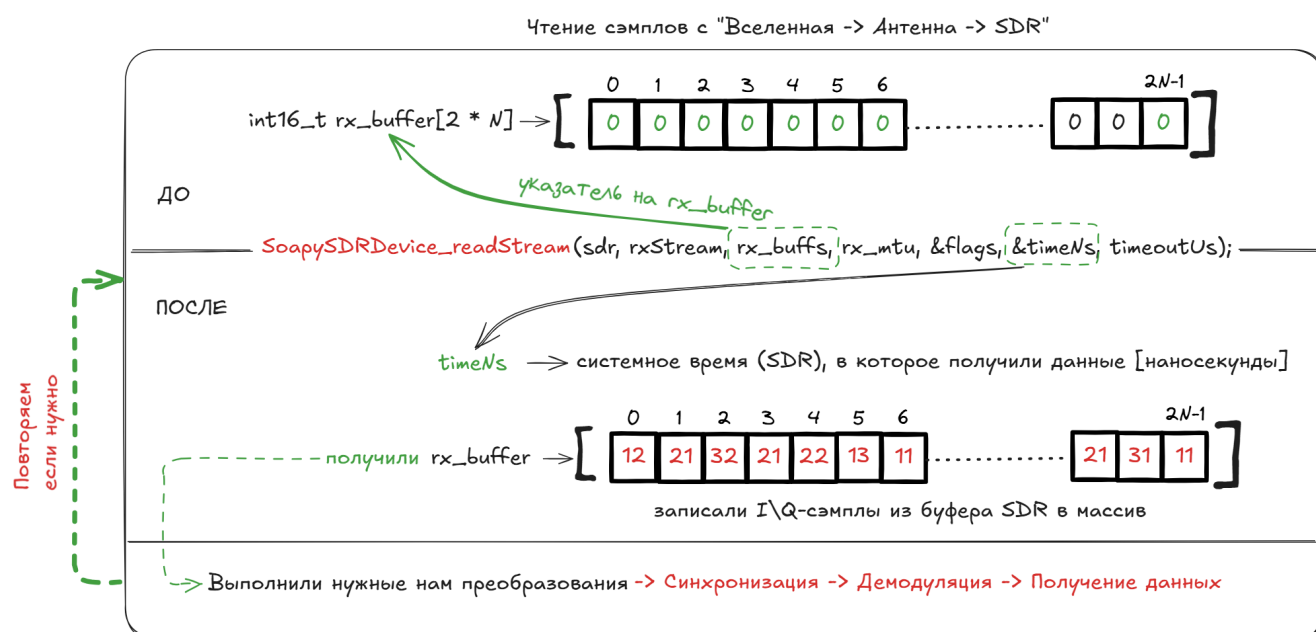


Рис. 15 — Чтение сэмплов

Выполнение

Сперва я заполнил буферы значениями 1500×4 ($1500 \times 2^4 = 24000$) Передавал их на SDR и смотрел что получается. Визуализировать удобно с помощью кода на python

Листинг 3.1 — Код для визуализации полученных данных с SDR

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 rx = np.fromfile("/home/plutoSDR/sdr/pluto/dev/rx.pcm", dtype=np.int16)
```



```

5
6 samples = []
7 for x in range(0, len(rx), 2):
8     samples.append(rx[x] + 1j * rx[x+1])
9
10 samples = np.array(samples)
11
12 ampl = np.abs(samples)
13 phase = np.angle(samples)
14 time = np.arange(len(samples))
15
16 plt.figure(figsize=(10, 8))
17
18 plt.subplot(3, 1, 1)
19 plt.plot(time, ampl)
20 plt.title("Amplitude")
21 plt.ylabel("Amplitude")
22 plt.grid(True)
23
24 plt.subplot(3, 1, 2)
25 plt.plot(time, phase)
26 plt.title("Phase")
27 plt.ylabel("Phase (rad)")
28 plt.grid(True)
29
30 plt.subplot(3, 1, 3)
31 plt.plot(time, np.real(samples), label='I (Real)')
32 plt.plot(time, np.imag(samples), label='Q (Imag)')
33 plt.title("I and Q components")
34 plt.ylabel("Value")
35 plt.xlabel("Time (samples)")
36 plt.legend()
37 plt.grid(True)
38
39 plt.tight_layout()
40 plt.show()

```

На графике получаем, амплитуду, фазу и I,Q части

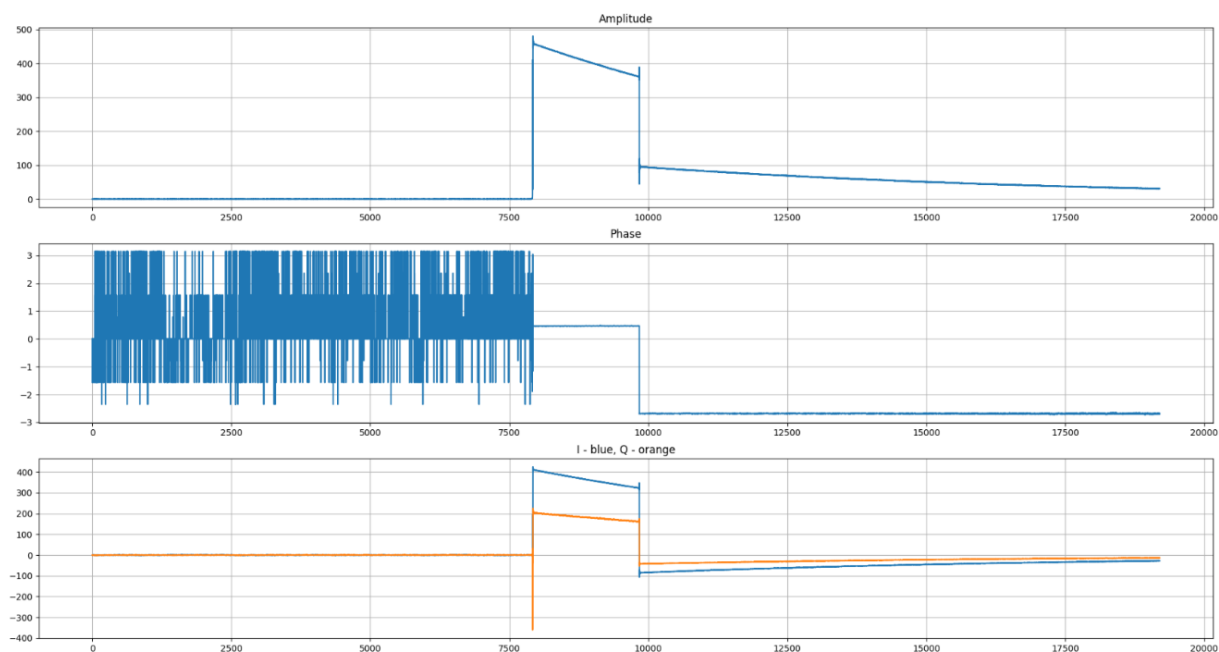


Рис. 16 — Графики

Вывод

Самостоятельно записали произвольные данные в буферы, который потом отправили на SDR, получили их, вывели на графики, с помощью python кода, наглядно показано, что получаем на выходе после всех этапов отправки данных.

4 ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ ПРИЕМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО ПРИНЯТЫХ ОТСЧЕТОВ СИГНАЛА (СЕМПЛОВ)

[Ссылка на GitHub](#)

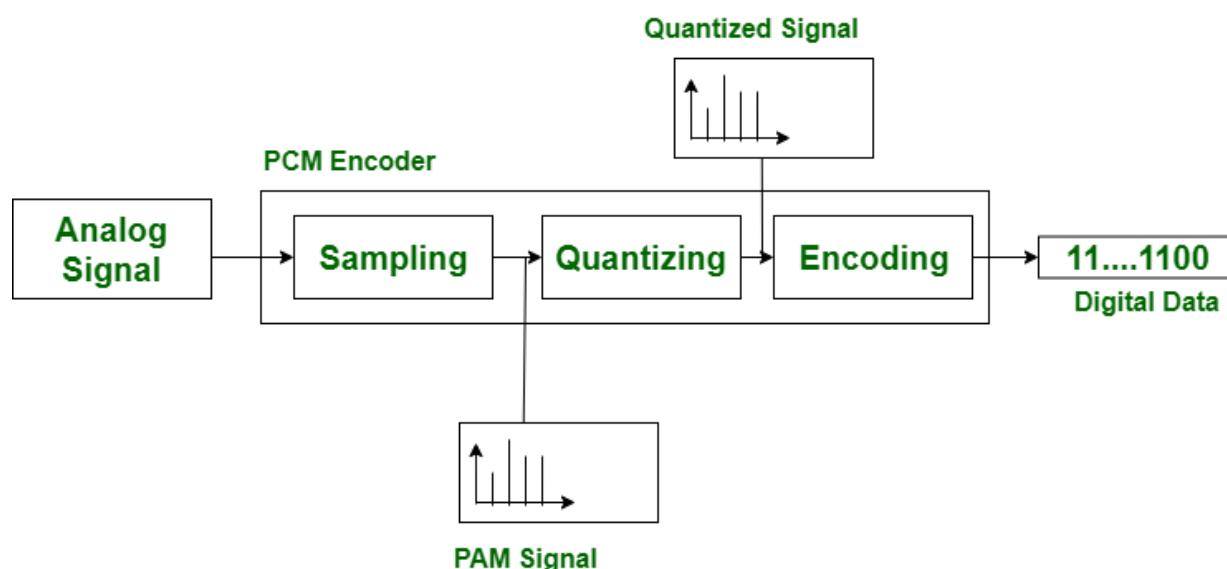
Цель практики:

Попробовать отправить свою аудио запись, получить ее и посмотреть что поменялось. Попробовать получить эффект сложения сигналов, путем скрещения антен двух Pluto SDR.

Краткие теоретические сведения

Сперва что такое MP3, PCM?

PCM (Pulse Code Modulation) — это линейное, несжатое представление звука. Сигнал записывается как последовательность отсчётов амплитуды с фиксированной частотой дискретизации и глубиной квантования. PCM хранит аудио без сжатия, что делает его максимально точным, но объёмным.



Block Diagram Of PCM

Рис. 17 — Графики

MP3 (MPEG-1 Audio Layer III) — это формат сжатия аудио с потерями, основанный на психоакустической модели. Он удаляет компоненты звука, менее заметные человеческому слуху, и применяет дополнительные методы кодирования, что существенно уменьшает размер файла.

Для конвертации между этими форматами можно использовать библиотеку pydub в Python. А для конвертации MP3 в WAV можно использовать ffmpeg.

Листинг 4.1 — Код на C++ для чтения PCM файла

```
1 int16_t *read_pcm(const char *filename, size_t *sample_count)
2 {
3     FILE *file = fopen(filename, "rb");
4
5     fseek(file, 0, SEEK_END);
6     long file_size = ftell(file);
7     fseek(file, 0, SEEK_SET);
8     printf("file_size = %ld\n", file_size);
9
10    *sample_count = file_size / sizeof(int16_t);
11    int16_t *samples = (int16_t *)malloc(file_size);
12    size_t sf = fread(samples, sizeof(int16_t), *sample_count, file);
13
14    if (sf == 0){
15        printf("file %s empty!", filename);
16    }
17
18    fclose(file);
19
20    return samples;
21 }
```

Выполнение

Для начала, я написал код для конвертации mp3 файла в pcm формат с помощью pydub.

Листинг 4.2 — Код на python для конвертации mp3 в pcm

```
1 import numpy as np
2 import librosa
3 from pydub import AudioSegment
4
5 def mp3_to_pcm(src_path, dst_path, sample_rate=44100):
6     audio_data, sr = librosa.load(src_path, sr=sample_rate, mono=False)
7     if audio_data.ndim > 1:
8         audio_data = audio_data.T.flatten()
9     pcm_array = np.int16(audio_data * 32767)
10    pcm_array.tofile(dst_path)
11
12 def pcm_to_mp3(pcm_path, output_path, channels=2, sample_rate=44100, bitrate="160k"):
13     raw_audio = np.fromfile(pcm_path, dtype=np.int16)
14     sound = AudioSegment(
15         data=raw_audio.tobytes(),
16         sample_width=2,
```

```

17     frame_rate=sample_rate,
18     channels=channels
19 )
20 sound.export(output_path, format="mp3", bitrate=bitrate)
21
22 print("""PCM->MP3: 0
23 MP3->PCM: 1""")
24 ans = int(input("Выберите режим: "))
25 if ans == 1:
26     mp3_to_pcm("../1.mp3", "../1.pcm")
27 else:
28     pcm_to_mp3("../2.pcm", "../2.mp3")

```

Он на выбор преобразует mp3 файл в pcm или наоборот. Используем его перед началом экспериментов. В main.cpp я добавил чтение pcm файла и заполнение буфера прочитанными значениями.

Листинг 4.3 — Заполнения буфера данными из pcm

```

1 size_t iteration_count = 10;
2 long long last_time = 0;
3
4 FILE* file = fopen("../rx.pcm", "wb");
5 FILE* file1 = fopen("../tx.pcm", "wb");
6
7 for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
8 {
9     void *rx_buffs[] = {rx_buffer};
10    int flags;           // flags set by receive operation
11    long long timeNs;    //timestamp for receive buffer
12    long timeoutUs = 1000000;
13    int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
        timeNs, timeoutUs);
14
15    printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
        buffers_read, sr, flags, timeNs, timeNs - last_time);
16    last_time = timeNs;
17
18    long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс[] вбудущее
19
20    for(size_t i = 0; i < 8; i++)
21    {
22        uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
23        tx_buff[2 + i] = tx_time_byte << 4;
24    }
25
26    void *tx_buffs[] = {tx_buff};
27    if( (buffers_read==2) ){
28        flags = SOAPY_SDR_HAS_TIME;
29        int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
        tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);

```

```

30         if ((size_t)st != tx_mtu)
31         {
32             printf("TX Failed: %i\n", st);
33         }
34     }
35
36     fwrite(rx_buffer, 2 * rx_mtu * sizeof(int16_t), 1, file);
37     fwrite(tx_buffs, 2 * rx_mtu * sizeof(int16_t), 1, file1);
38 }
39
40 fclose(file);

```

Не забываем закрыть файл через `fclose`, и получается, что мы записали и отправили наш pcm файл через Pluto SDR, `rx_buffer` в `file(rx.pcm)` и `tx_buffer` в `file1(tx.pcm)`. С помощью нашего кода на python мы можем конвертировать pcm обратно в mp3, чтобы прослушать, полученные mp3 получились искаженными. Если скрестить две Pluto SDR, одновременно начать отправку сэплов, то получится эффект наложения сигналов. У нас с одноклассником получилось так, что одновременно были слышны 2 песни.

Вывод

Я научился переводить файлы из mp3 в pcm и обратно, записывать их в буфер Pluto SDR, отправлять их и получать, полученные файлы были конвертированы обратно в mp3 для того чтобы прослушать, что получилось передать.

5 ДИСКРЕТНАЯ СВЕРТКА. РЕАЛИЗАЦИЯ ПРИЕМА И ПЕРЕДАЧИ BPSK СИГНАЛОВ

I [Ссылка на GitHub](#)

Цель практики:

Программно реализовать дискретную свертку, используя язык программирования C++, передать полученные после свертки биты, посмотреть, что мы получим на выходе

Краткие теоретические сведения

Дискретная свертка — это математическая операция над двумя дискретными последовательностями (сигналами), определяемая следующим образом:

Пусть даны две последовательности $x[n]$ и $h[n]$, где $n \in \mathbb{Z}$. Их свертка $y[n]$ вычисляется по формуле:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

Для последовательностей конечной длины L и M , где $x[n]$ определена при $0 \leq n \leq L - 1$, а $h[n]$ — при $0 \leq n \leq M - 1$, результирующая последовательность $y[n]$ будет иметь длину $N = L + M - 1$ и формула принимает вид:

$$y[n] = \sum_{k=\max(0, n-M+1)}^{\min(n, L-1)} x[k] \cdot h[n - k], \quad \text{для } 0 \leq n \leq N - 1$$

Свойства дискретной свертки:

- **Коммутативность:** $x * h = h * x$
- **Ассоциативность:** $(x * h) * g = x * (h * g)$
- **Дистрибутивность:** $x * (h + g) = (x * h) + (x * g)$

Передавать мы будем биты после свертки. Перед этим биты нужно удлинить, за это будет отвечать функция `upsample`, которая из одного бита делает например 10. То есть у нас было `[1011]` битов, а стало (если `upsample` до 5) `[10000,00000,10000,10000]`, далее эти биты идут в функцию под названием `filter`, которая выполняет свертку с единичным импульсом после которой у нас получается такой результат `[11111,00000,11111,11111]`. Получается, что мы продлеваем наш сигнал.

BPSK и QPSK

BPSK (Binary Phase Shift Keying) — это цифровая модуляция, при которой двоичные символы 0 и 1 передаются изменением фазы несущего сигнала на 0 и π радиан соответственно. Сигнал BPSK можно записать как:

$$s_{\text{BPSK}}(t) = A \cos(2\pi f_c t + \phi), \quad \phi = \begin{cases} 0, & \text{для символа 1,} \\ \pi, & \text{для символа 0.} \end{cases}$$

QPSK (Quadrature Phase Shift Keying) — это модуляция, передающая два бита за символ с использованием четырёх фазовых состояний, разнесённых на $\frac{\pi}{2}$ радиан. Сигнал QPSK имеет вид:

$$s_{\text{QPSK}}(t) = A \cos\left(2\pi f_c t + \frac{(2k+1)\pi}{4}\right), \quad k = 0, 1, 2, 3,$$

что соответствует фазам $\frac{\pi}{4}, \frac{3\pi}{4}, \frac{5\pi}{4}, \frac{7\pi}{4}$.

BPSK обеспечивает более высокую помехоустойчивость, тогда как QPSK удваивает спектральную эффективность по сравнению с BPSK при том же энергетическом расходе на бит.

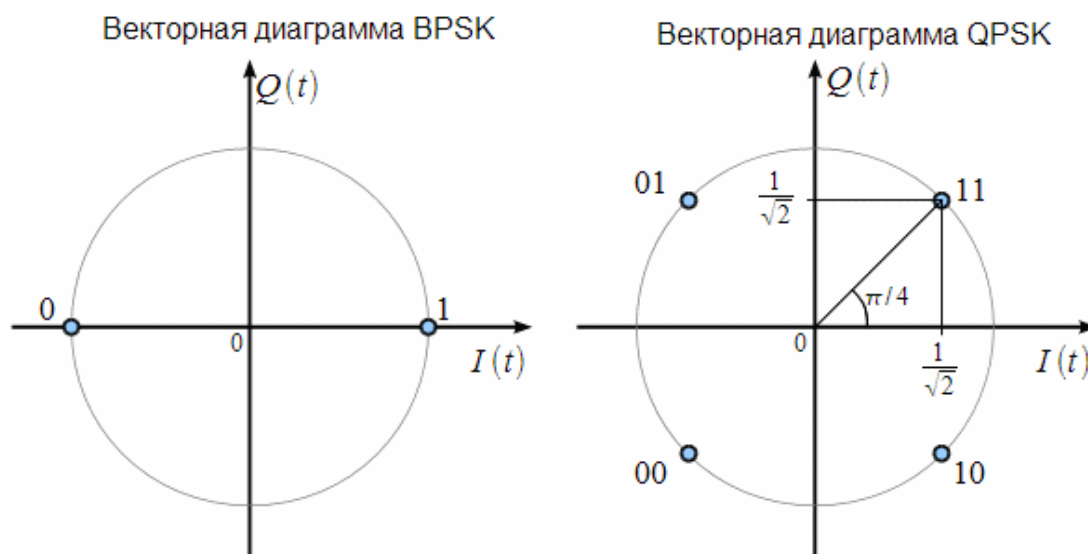


Рис. 18 — Векторная диаграмма BPSK, QPSK

На осциллографе QPSK сигнала, должно быть видно как меняется фаза

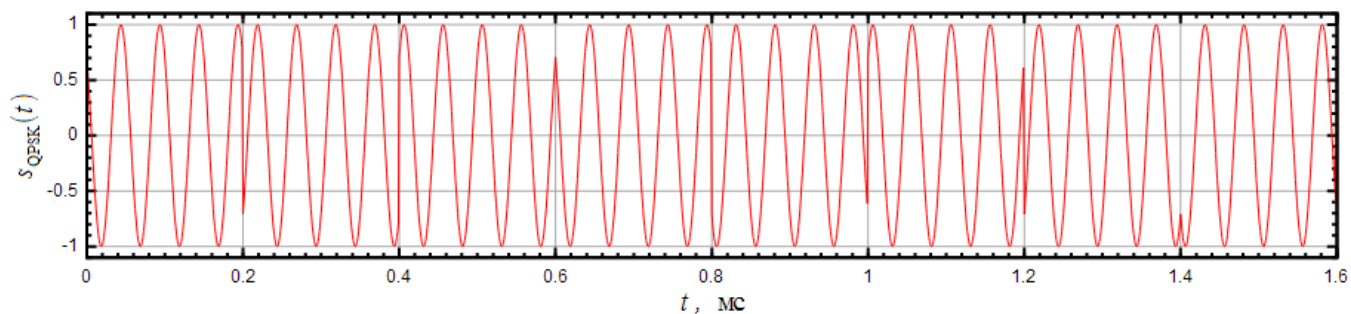


Рис. 19 — Осциллограмма QPSK сигнала

Структурная схема QPSK модулятора будет выглядеть так:

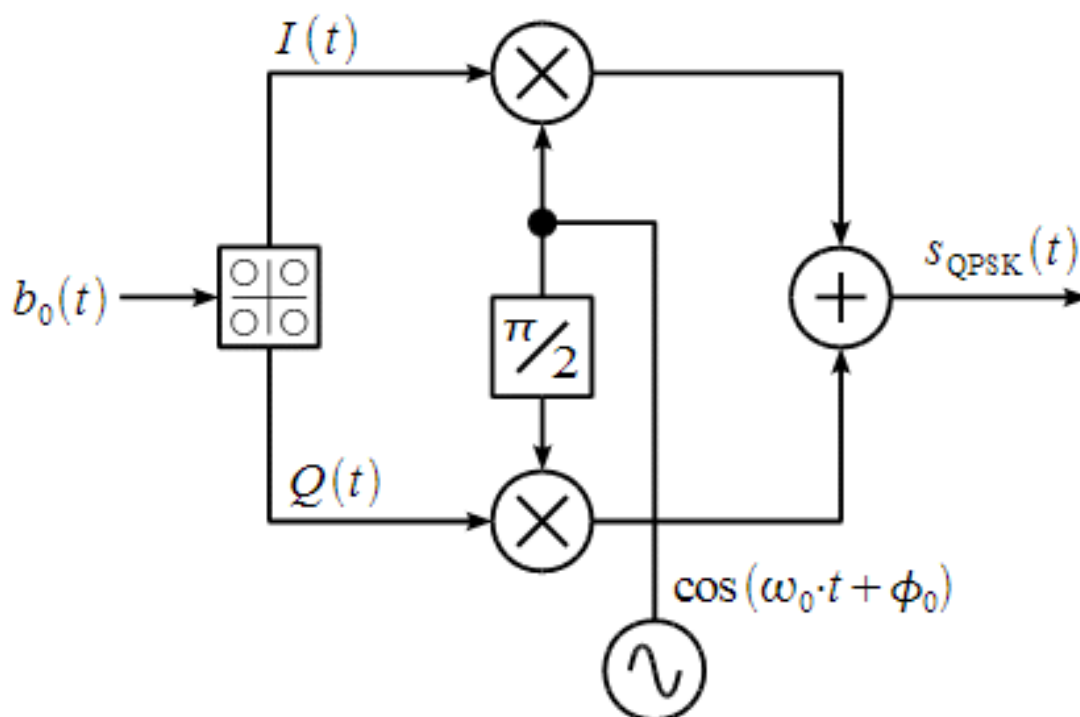


Рис. 20 — Структурная схема QPSK модулятора

Выполнение

Вместо BPSK, будем использовать QPSK. Напишем код для перевода в QPSK символы

Листинг 5.1 — map для QPSK

```
1 // QPSK символы
2 const complex<double> i0 (0, 0);
3 const complex<double> i1 (1, 1);
4 const complex<double> i2 (-1, 1);
5 const complex<double> i3 (-1, -1);
6 const complex<double> i4 (1, -1);
```

```

7
8 static map<string, complex<double>> qpsk_map = {
9     {"00", i1},
10    {"01", i2},
11    {"11", i3},
12    {"10", i4}
13 };

```

Функция для UpSampler, которая удлиняет сигнал, реализована с помощью простых циклов

Листинг 5.2 — UpSampler

```

1 void UpSampler(complex<double> *symbols, int len_s, complex<double> *symbols_ups, int
  L){
2     for (size_t i = 0; i < (size_t)len_s*(size_t)L; i++) symbols_ups[i] = i0;
3     for (size_t i = 0; i < (size_t)len_s; i++) symbols_ups[i*L] = symbols[i];
4 }

```

Функция для свертки, называется в моем коде filter, реализует процесс свертки массива уже после upsample с еденичным импульсом с заданной нами длиной

Листинг 5.3 — filter

```

1 void filter(complex<double> *symbols_ups, int len_symbols_ups, complex<double> *
  impulse, int L) {
2     vector<complex<double>> sum(len_symbols_ups, 0);
3     for (size_t i = 0; i < (size_t)len_symbols_ups; i++) {
4         for (size_t j = 0; j < (size_t)L && (int)(i-j) >= 0; j++) {
5             sum[i] += impulse[j] * symbols_ups[i-j];
6         }
7     }
8     memcpy(symbols_ups, sum.data(), len_symbols_ups * sizeof(complex<double>));
9 }

```

Функция Mapper, которая формирует из 2 битов один символ, который далее соотносится к соотвующему комплексному значению в нашем qpsk_map.

Листинг 5.4 — Mapper

```

1 void Mapper(int16_t *bits, int len_b, complex<double> *symbols){
2     string pair_bits;
3     for (size_t i = 0; i < (size_t)len_b; i += 2){
4         pair_bits = to_string(bits[i]) + to_string(bits[i+1]);
5         symbols[i/2] = qpsk_map[pair_bits];
6     }
7 }

```

Для практики я случайно формирую 20 битов от 0 до 1, затем пропускаю их через Mapper для того чтобы сформировать из битов символы по 2 бита каждый, далее пропускаю эти символы через UpSampler чтобы ”продлить” сигнал, затем подаю уже удлиненные символы на filter, который делает свертку с еденичным импульсом

Листинг 5.5 — Как это устроено в коде

```

1 int n = 20;
2 int16_t *bits = (int16_t*)malloc(n * sizeof(int16_t));
3 for (int i = 0; i < n; i++) bits[i] = rand() % 2;
4
5 int len_symbols = n / 2;
6 complex<double> *symbols = (complex<double>*)malloc(len_symbols * sizeof(complex<
    double>));
7 int L = 10;
8 int len_symbols_ups = len_symbols * L;
9 complex<double> *symbols_ups = (complex<double>*)malloc(len_symbols_ups * sizeof(
    complex<double>));
10 complex<double> impulse[L];
11 for (int i = 0; i < L; i++) impulse[i] = 1.0;
12
13 Mapper(bits, n, symbols);
14 UpSampler(symbols, len_symbols, symbols_ups, L);
15 filter(symbols_ups, len_symbols_ups, impulse, L);
16 // Show_ArrayСИМВОЛЫ("", symbols_ups, len_symbols_ups);
17
18 int16_t *tx_samples = (int16_t*)malloc(2 * config.tx_mtu * sizeof(int16_t));
19 for (size_t i = 0; i < (size_t)config.tx_mtu; i++) {
20     tx_samples[2*i] = (int16_t)((real(symbols_ups[i])) * 1000) << 4;
21     tx_samples[2*i + 1] = (int16_t)((imag(symbols_ups[i])) * 1000) << 4;
22 }
23
24 // memcpy(config.tx_buff, tx_samples, 2 * len_symbols_ups * sizeof(int16_t));
25 fwrite(tx_samples, sizeof(int16_t), 2 * len_symbols_ups, tx);

```

Так же для того чтобы увидеть наши биты нам нужен код на python, который будет визуализировать все, что мы получили

Листинг 5.6 — Визуализация полученных битов

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 print("""Choose 0 - Tx
5 Choose 1 - Rx""")
6
7 answer = int(input())
8
9 if answer == 1:
10     nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np
        .int16)
11 else:
12     nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np
        .int16)
13

```

```

14 rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)
15 tx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np.
    int16)
16 print(f"RX: {rx}\nTX: {tx}")
17
18 samples = []
19
20 for x in range(0, len(nx), 2):
21     samples.append(nx[x] + 1j * nx[x+1])
22
23 ampl = np.abs(samples)
24 phase = np.angle(samples)
25 time = np.arange(len(samples))
26
27 # plt.subplot(3,1,1)
28 # plt.plot(time, ampl)
29 # plt.grid(True)
30 # plt.tight_layout()
31 # plt.title("Amplitude")
32
33 # plt.subplot(3,1,2)
34 # plt.plot(time, phase)
35 # plt.grid(True)
36 # plt.tight_layout()
37 # plt.title("Phase")
38
39
40 # plt.subplot(3,1,3)
41 plt.plot(time, nx[0::2])
42 # plt.plot(time, nx[1::2])
43 plt.grid(True)
44 plt.tight_layout()
45 plt.title("Real and Imag parts")
46 plt.show()

```

По итогу мы можем считать наши `rx` файлы, в которых содержатся наши сгенерированные биты от 0 до 1, можем вывести их амплитуду, фазу и взять реальную и мнимую части

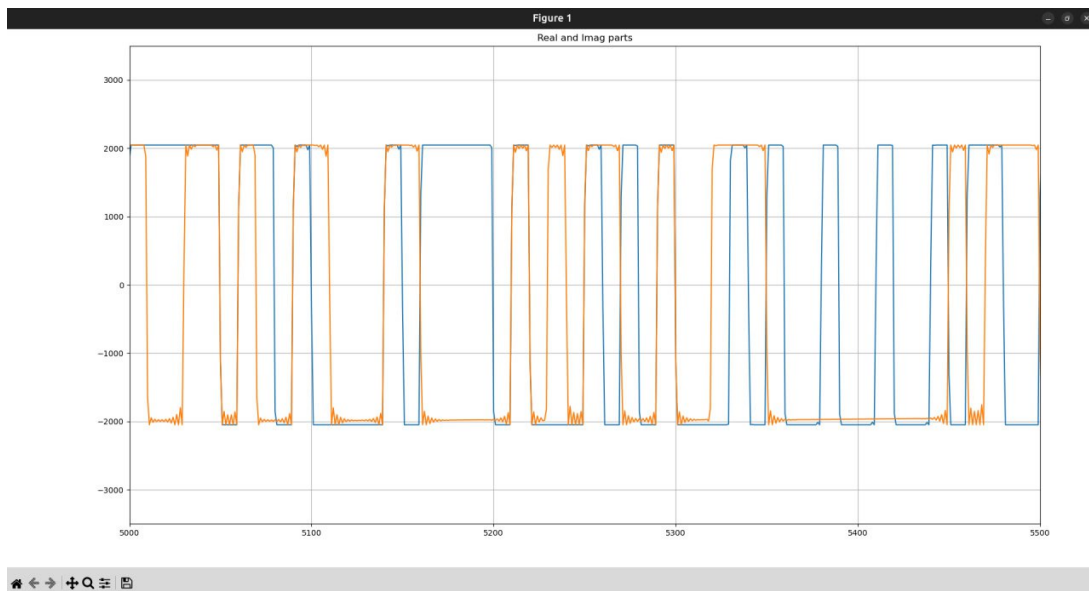


Рис. 21 — Реальная и мнимая части полученных битов

Вывод

Получилось отправить и принять, сгенерированные биты, которые прошли через Mapper, UpSampler, filter, отобразить их амплитуду, фазу, реальную и мнимую части, получить графики.

6 ПРИЕМ QPSK BPSK, ПРИЕМ НА СОГЛАСОВАННЫЙ ФИЛЬТР, ГЛАЗКОВАЯ ДИАГРАММА, ПОИСК ОПТИМАЛЬНОГО ОТСЧЕТНОГО ЗНАЧЕНИЯ И НЕОБХОДИМОСТЬ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ

Ссылка на GitHub

Цель практики:

Сформировать биты, пропустить их через согласованный фильтр. Построить глазковую диаграмму, вручную найти оптимальное отсчетное значение

Краткие теоретические сведения

Согласованный фильтр (matched filter) в цифровой обработке сигналов — это линейный фильтр, обеспечивающий максимальное отношение сигнал/шум (SNR) на выходе в момент принятия решения при наличии аддитивного белого гауссовского шума (AWGN).

Для известного сигнала $s(t)$ длительностью T , импульсная характеристика согласованного фильтра задаётся как:

$$h(t) = s(T - t),$$

то есть представляет собой зеркально отражённую во времени копию сигнала $s(t)$, сдвинутую на T .

В дискретной форме, для последовательности отсчётов $s[n]$, $n = 0, 1, \dots, N - 1$, импульсная характеристика согласованного фильтра:

$$h[n] = s^*[N - 1 - n],$$

где $(\cdot)^*$ обозначает комплексное сопряжение.

Выход согласованного фильтра вычисляется как свёртка входного сигнала $x[n]$ с $h[n]$:

$$y[n] = \sum_{k=0}^{N-1} x[k] \cdot s^*[k - n],$$

что эквивалентно корреляции входного сигнала с ожидаемой формой сигнала.

Согласованный фильтр широко применяется в программно-определяемых радиосистемах (SDR) для обнаружения и демодуляции сигналов с известной структурой (например, пилотных символов, преамбулы или кодов расширения в CDMA).

Глазковая диаграмма (eye diagram) — это визуальное представление цифрового сигнала, полученное путём наложения друг на друга нескольких последовательных тактовых интервалов в осциллографе или программной среде.

Формируется глазковая диаграмма следующим образом: осциллограф или программа захватывает отрезки сигнала длительностью, кратной периоду тактовой частоты T , и отображает их синхронно по фронту тактового сигнала. В результате наложения траекторий сигнала возникает характерная форма, напоминающая «глаз».

Качество цифровой передачи оценивается по степени раскрытия «глаза»: - чем шире и выше глаз, тем меньше межсимвольная интерференция (ISI) и шум; - закрытый глаз указывает на сильную ISI, джиттер или высокий уровень шума.

Глазковая диаграмма (eye diagram) дает наглядное представление о форме принимаемого сигнала на выходе согласованного фильтра. Глазковая диаграмма строится в виде накапливающего изображения сигналов на выходе СФ в интервале времени

$$0.5T_s < t < 0.5T_s$$

по большому числу символов

Если изобразить много таких отрезков сигнала на выходе СФ в виде накапливающейся суммы изображений, то получим глазковую диаграмму.

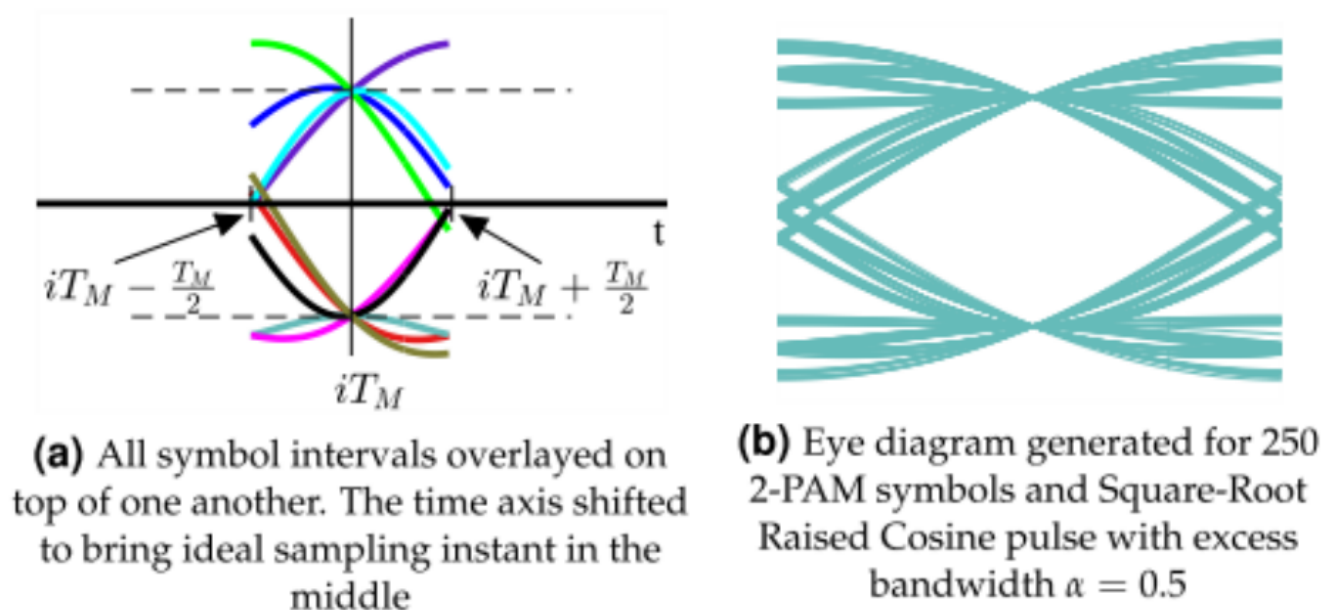


Рис. 22 — Глазковая диаграмма

Выполнение

Согласованный фильтр уже реализован в функции `filter`

Листинг 6.1 — `filter`

```
1 void filter(complex<double> *symbols_ups, int len_symbols_ups, complex<double> *
  impulse, int L) {
2     vector<complex<double>> sum(len_symbols_ups, 0);
3     for (size_t i = 0; i < (size_t)len_symbols_ups; i++) {
```

```

4         for (size_t j = 0; j < (size_t)L && (int)(i-j) >= 0; j++) {
5             sum[i] += impulse[j] * symbols_ups[i-j];
6         }
7     }
8     memcpy(symbols_ups, sum.data(), len_symbols_ups * sizeof(complex<double>));
9 }

```

Чтобы получить глазковую диаграмму, нужно написать наш код на python

Листинг 6.2 — python eye diagram

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 data = np.fromfile("/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)
5
6 samples = []
7 for x in range(0, len(data) - 1, 2):
8     samples.append(data[x] + 1j * data[x+1])
9
10 real = np.real(samples)
11
12 sps = 10
13 segments = []
14 for i in range(0, len(real) - 2*sps, sps):
15     segments.append(real[i:i + 2*sps])
16
17 plt.plot(np.array(segments).T)
18 plt.xlim(7.5, 12.5)
19 plt.grid(True)
20 plt.show()

```

Получим такой результат

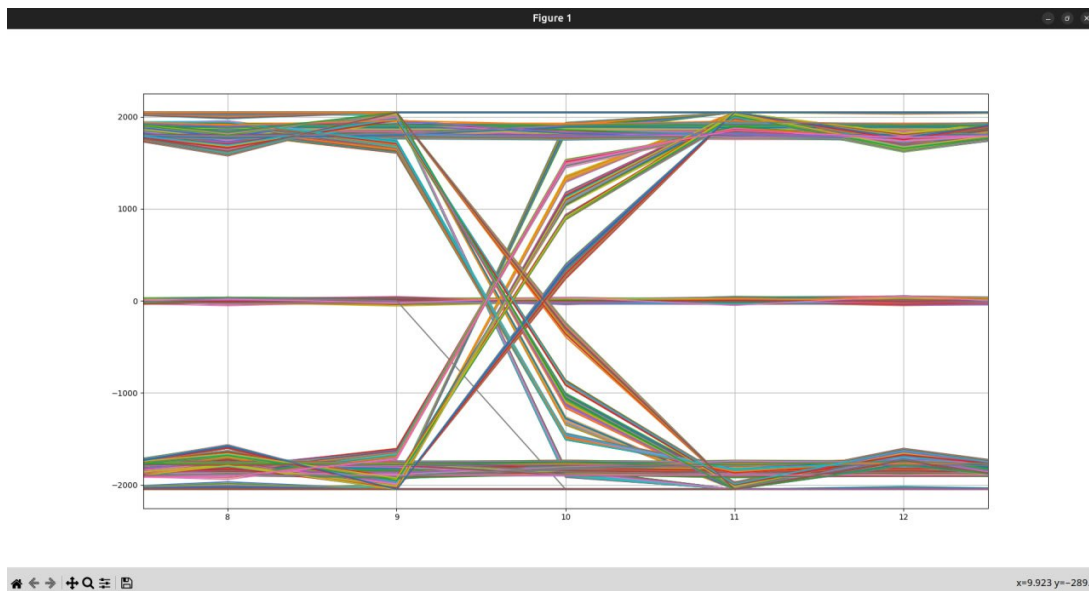


Рис. 23 — Глазковая диаграмма

Поиск оптимального значения отсчетного значения мы будем выполнять вручную. Сперва напишем код для отображения сигнального созвездия

Листинг 6.3 — python eye diagram

```

1 from matplotlib import pyplot as plt
2 import numpy as np
3
4 rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)
5
6 samples = []
7
8 for x in range(0, len(rx), 2):
9     samples.append((rx[x] + 1j * rx[x+1])/np.max(rx))
10
11 impulse = np.ones(10)
12
13 I = np.real(samples)
14 Q = np.imag(samples)
15
16 i_convolve = np.convolve(I, impulse)
17 q_convolve = np.convolve(Q, impulse)
18 i_con_dec = []
19 q_con_dec = []
20
21 for i in range(-1, len(i_convolve), 10):
22     i_con_dec.append(i_convolve[i])
23     q_con_dec.append(q_convolve[i])
24
25 # Убираем шум
26 i_con_dec = [x for x in i_con_dec if not (abs(x) <= 1)]

```

```

27 q_con_dec = [x for x in q_con_dec if not (abs(x) <= 1)]
28
29 plt.scatter(i_con_dec, q_con_dec)
30 plt.axhline()
31 plt.axvline()
32 plt.show()

```

Строчки когда я убираю шумы, иногда меняют длину массивов, что приводит к ошибке, поэтому иногда я их комментирую, и для таких значений у сигнального созвездия будет шум в центре

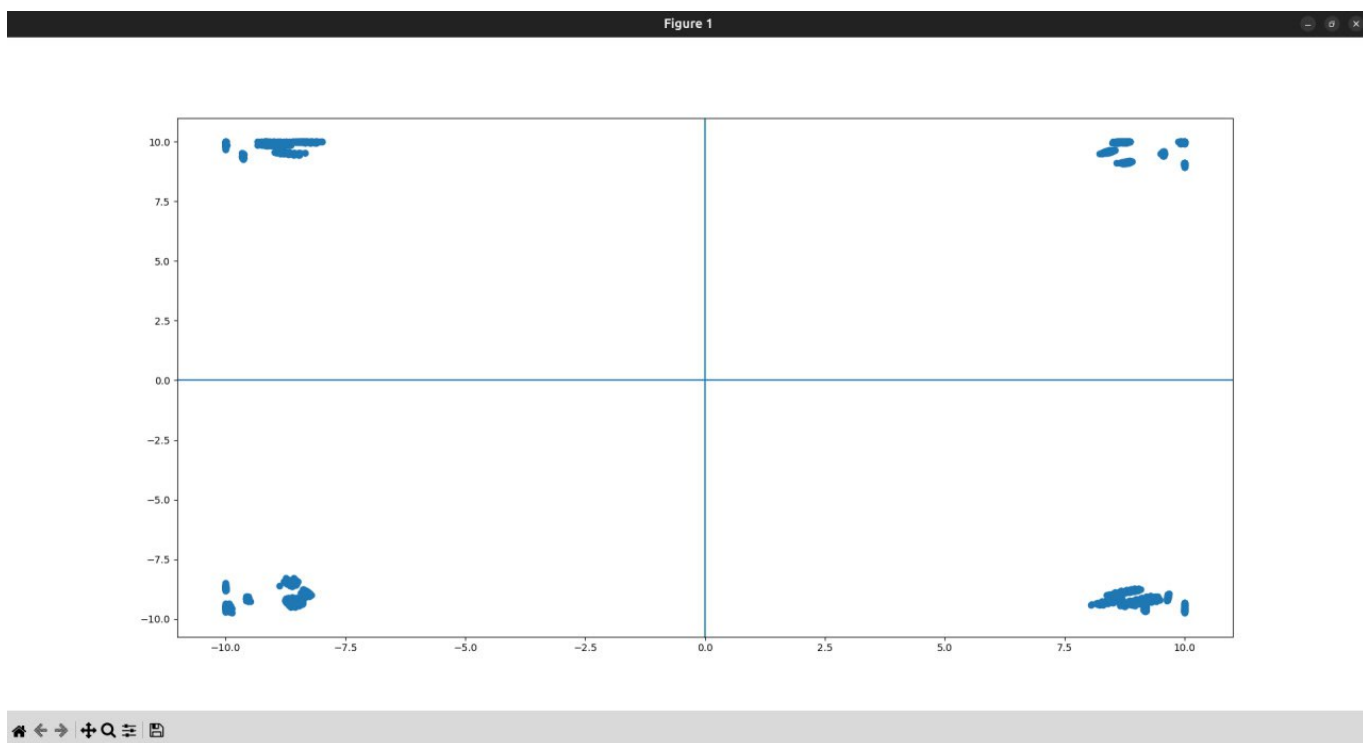


Рис. 24 — Отсчетное значение = -1

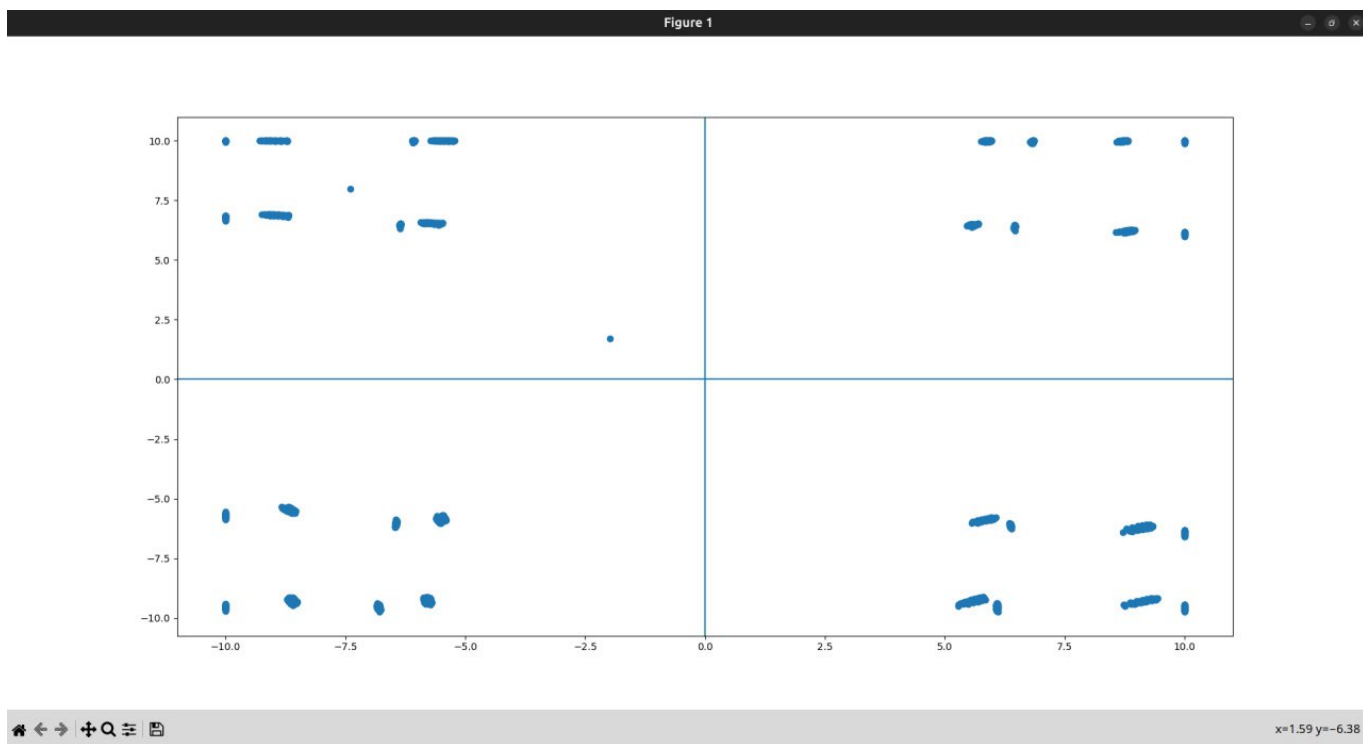


Рис. 25 — Отсчетное значение = 1

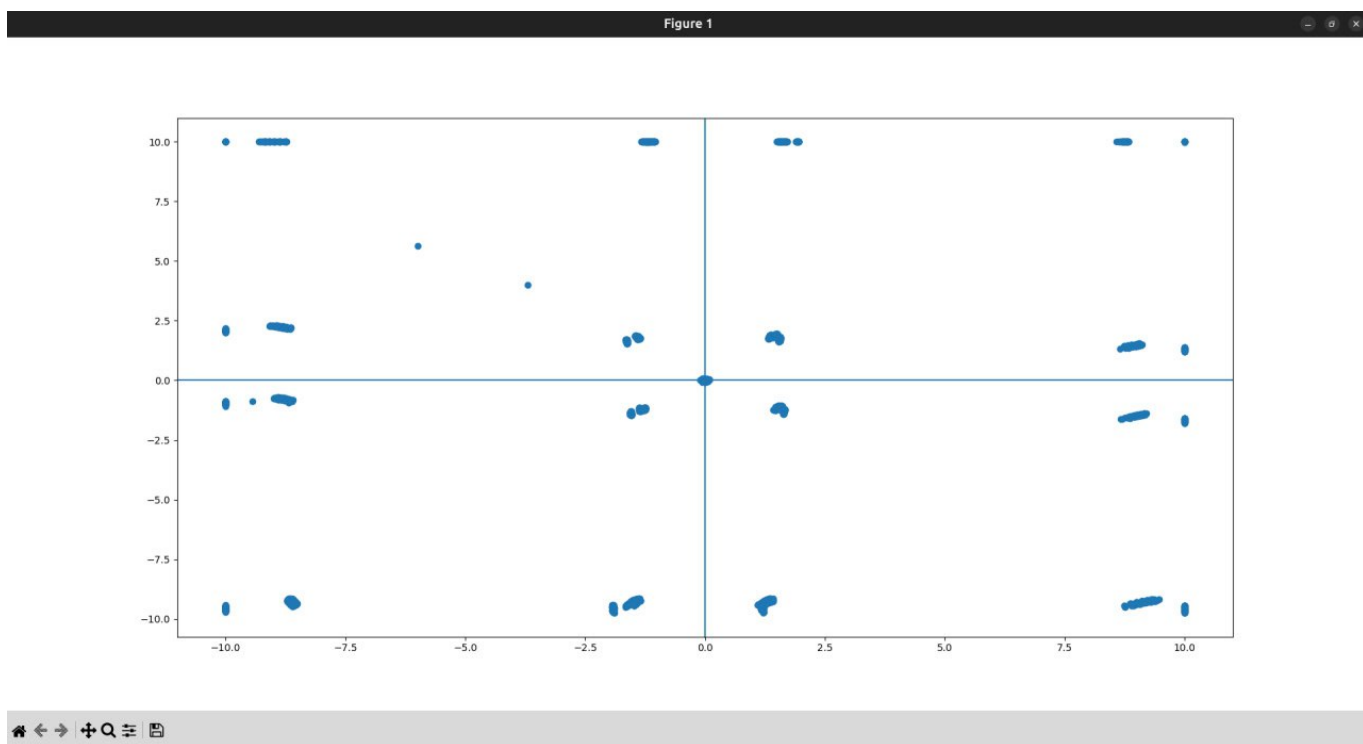


Рис. 26 — Отсчетное значение = 5

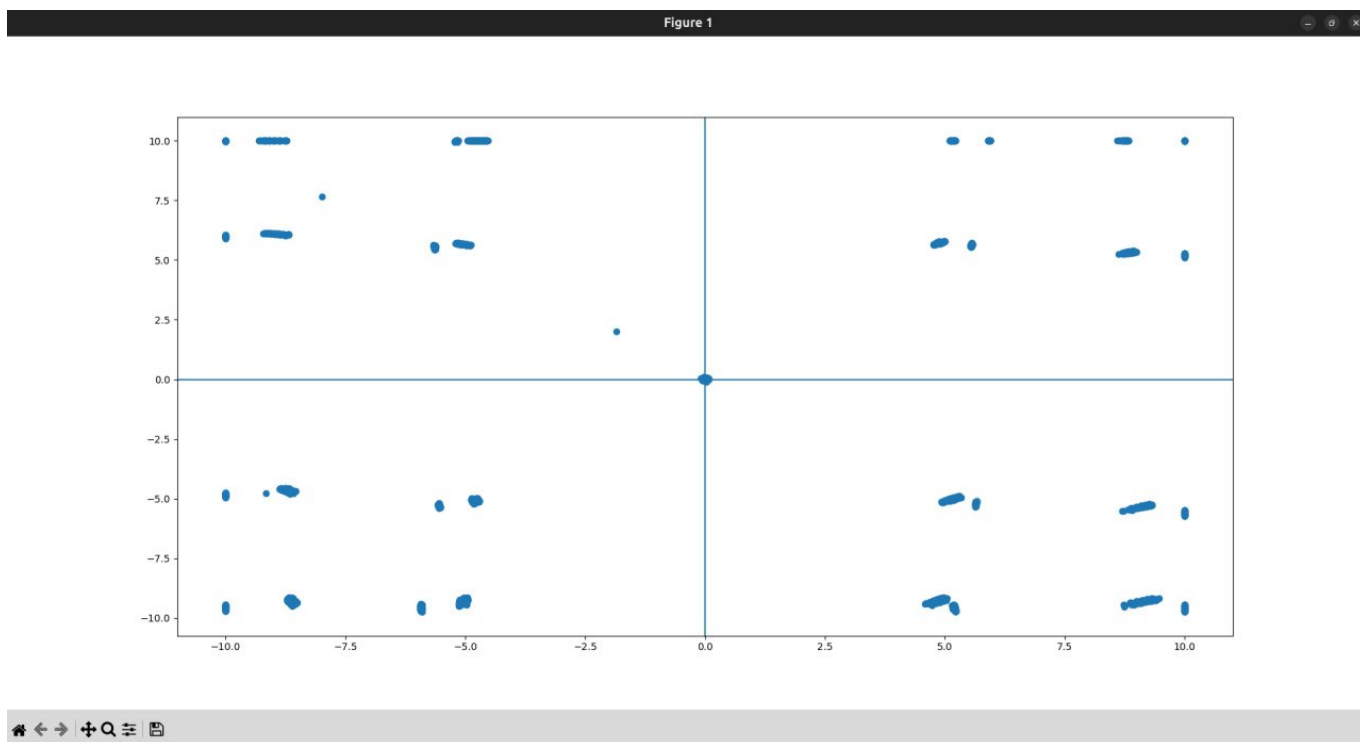


Рис. 27 — Отсчетное значение = 7

По итогу самое лучшее отсчетное значение оказалось -1, сигнальное созвездие с этим значением выглядит лучше всего, более похожее на сигнальное созвездие QPSK

Вывод

Получили глазковую диаграмму, в ручную подобрали отсчетные значения для лучшей схожести с сигнальным созвездием QPSK, получили графики для разных отсчетных значений.

7 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ(СИНХРОНИЗАЦИЯ ПРИЕМНИКА И ПЕРЕДАТЧИКА) НА SDR. НАПИСАНИЕ ФУНКЦИЙ ПЕТЛИ (КОНТУРА) СИНХРОНИЗАЦИИ

I [Ссылка на GitHub](#)

Цель практики:

Написать функцию которая будет автоматически синхронизировать примник и передатчик.

Краткие теоретические сведения

Оптимальный приемник выполняет прием дискретных сигналов с минимальной вероятностью ошибки. Строится оптимальный приемник по схеме на основе корреляторов или согласованных фильтров.

Оптимальный приемник выбирает наиболее близкое принятому сигналу $r(t)$ из набора возможных колебаний $s_i(t)$. Это может выполняться вычислением расстояния

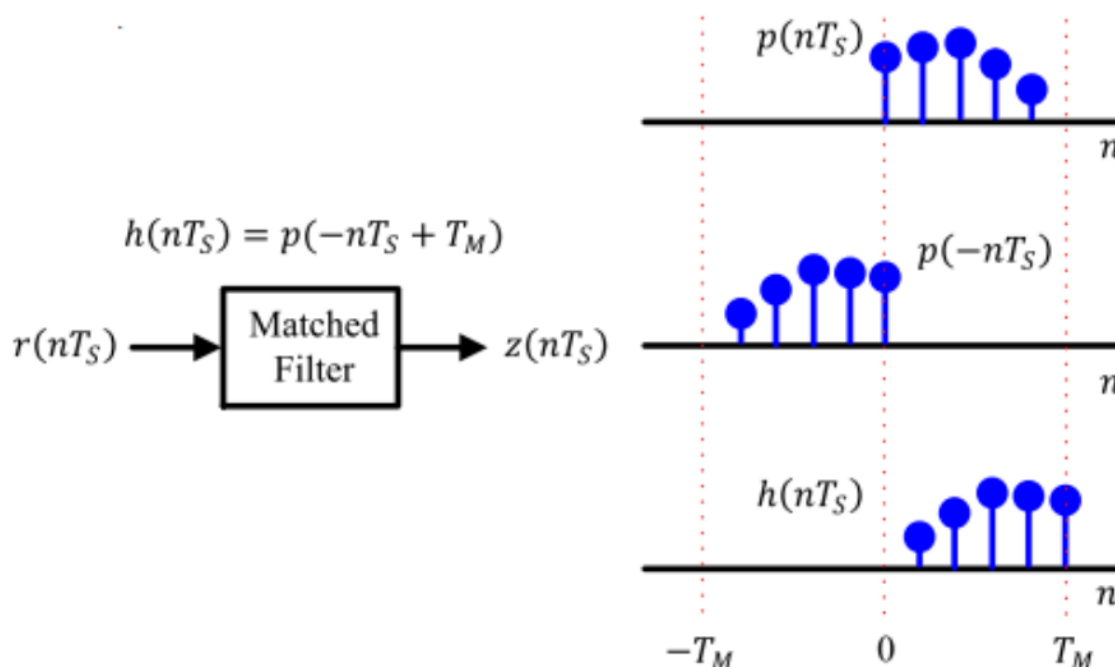


Рис. 28 — Matched filter

Если использовать формирующий фильтр с прямоугольной формой ИХ, то форма ИХ согласованного фильтра также является прямоугольной. Сигнал на выходе СФ вычисляется как свертка отсчетов входного сигнала и ИХ согласованного фильтра.

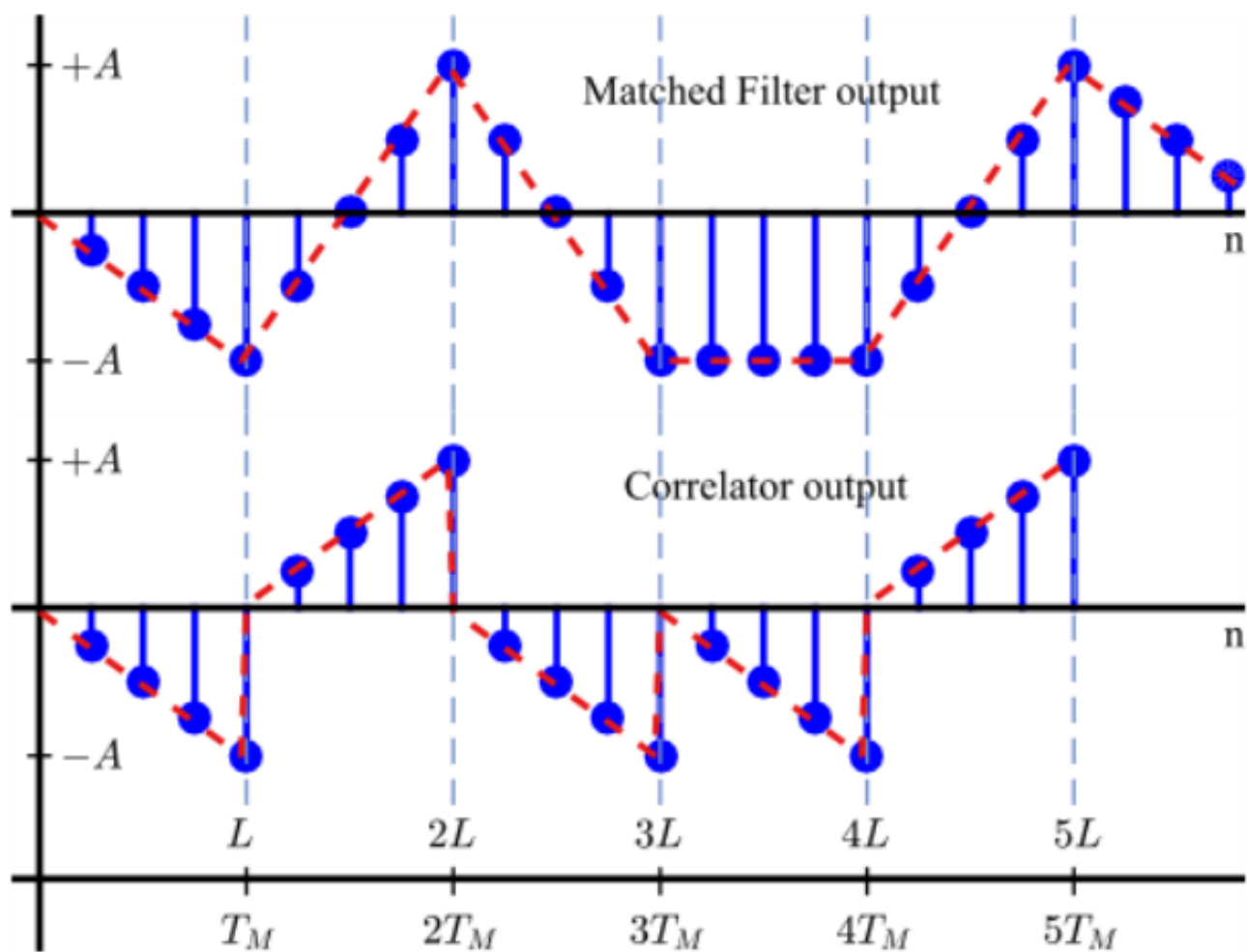


Рис. 29 — Matched filter/correlator output

Демодулятор должен сделать отсчет сигнала на выходе СФ точно в момент максимума сигнала на выходе СФ. Если время взятия отсчета не совпадает с максимумом, то невозможно принять надежное решение о передаваемом символе.

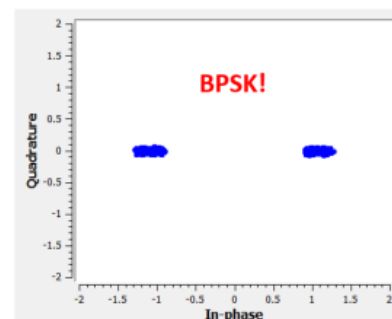
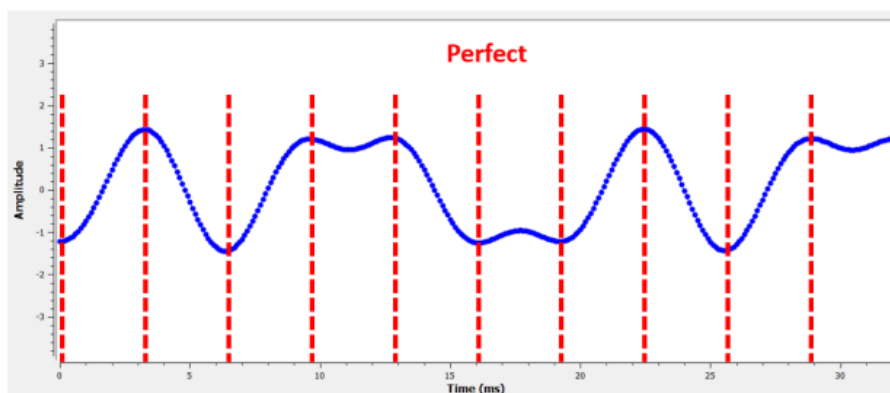


Рис. 30 — Идеальный отсчет

Время взятия отсчета (symbol timing) совпадает с максимумом сигнала на выходе СФ. При смещении времени взятия отсчета берутся значения с выхода СФ не в оптимальные моменты времени, что приводит к нарушению возможности точного определения амплитуды символа.

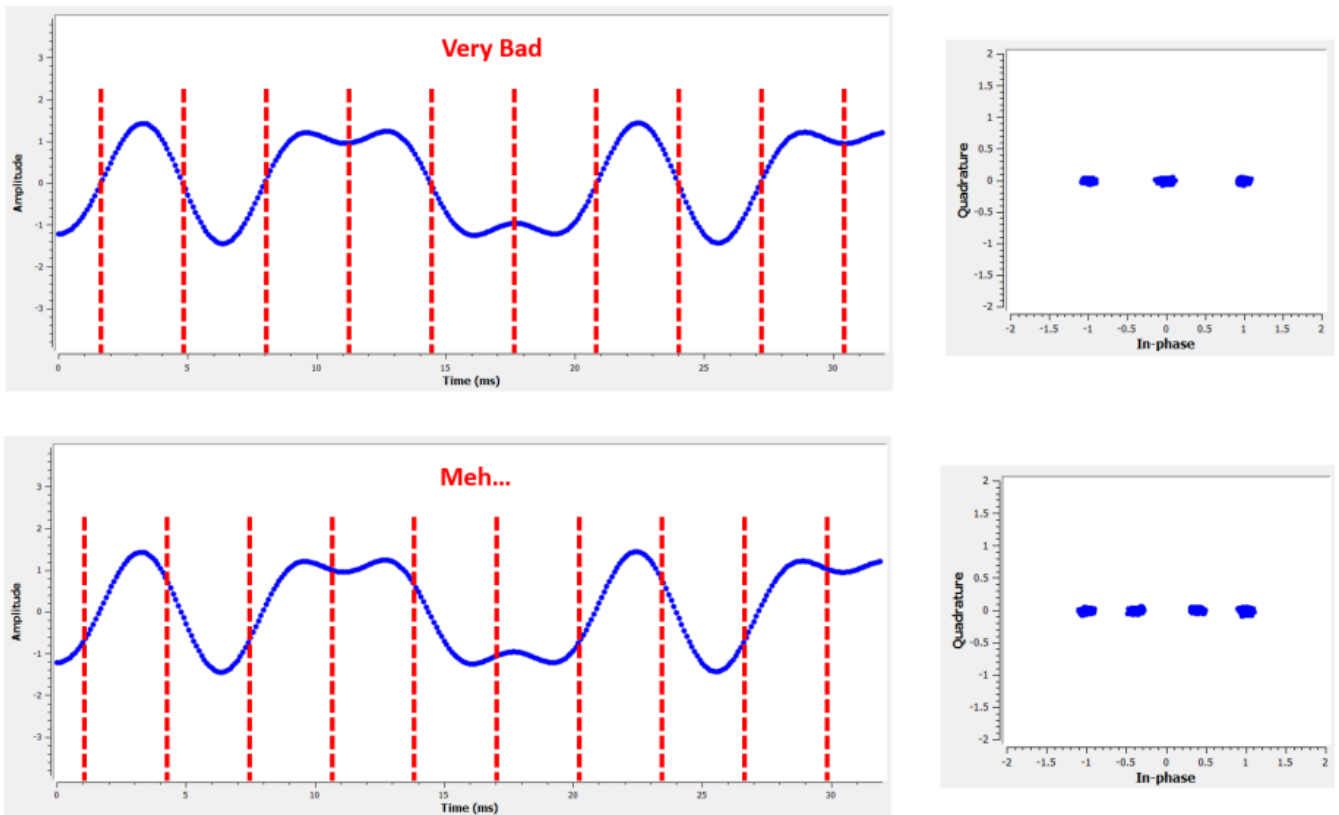


Рис. 31 — Плохой отсчет

Детектор временной ошибки - TED

Для поиска нужного отсчета сначала вычисляется ошибка положения символа при помощи детектора временной ошибки. Детектор использует набор отсчетов и вычисляет ошибку ТЕД $e(n)$, величина которой зависит от смещения максимума выхода СФ. В таблице приведены выражения для вычисления величины ошибки по отсчетам сигнала с выходы СФ.

TED Method	Expression
Zero-crossing (decision-directed)	$e(k) = x((k - 1/2)T_s + \hat{\tau})[\hat{a}_0(k - 1) - \hat{a}_0(k)] + y((k - 1/2)T_s + \hat{\tau})[\hat{a}_1(k - 1) - \hat{a}_1(k)]$
Gardner (non-data-aided)	$e(k) = x((k - 1/2)T_s + \hat{\tau})[x((k - 1)T_s + \hat{\tau}) - x(kT_s + \hat{\tau})] + y((k - 1/2)T_s + \hat{\tau})[y((k - 1)T_s + \hat{\tau}) - y(kT_s + \hat{\tau})]$
Early-late (non-data-aided)	$e(k) = x(kT_s + \hat{\tau})[x((k + 1/2)T_s + \hat{\tau}) - x((k - 1/2)T_s + \hat{\tau})] + y(kT_s + \hat{\tau})[y((k + 1/2)T_s + \hat{\tau}) - y((k - 1/2)T_s + \hat{\tau})]$
Mueller-Muller (decision-directed)	$e(k) = \hat{a}_0(k - 1)x(kT_s + \hat{\tau}) - \hat{a}_0(k)x((k - 1)T_s + \hat{\tau}) + \hat{a}_1(k - 1)y(kT_s + \hat{\tau}) - \hat{a}_1(k)y((k - 1)T_s + \hat{\tau})$

Рис. 32 — TED

Зависимость сигнала на выходе TED от величины смещения символа (временной ошибки) называется S-curve, для детектора Гарднера она имеет форму

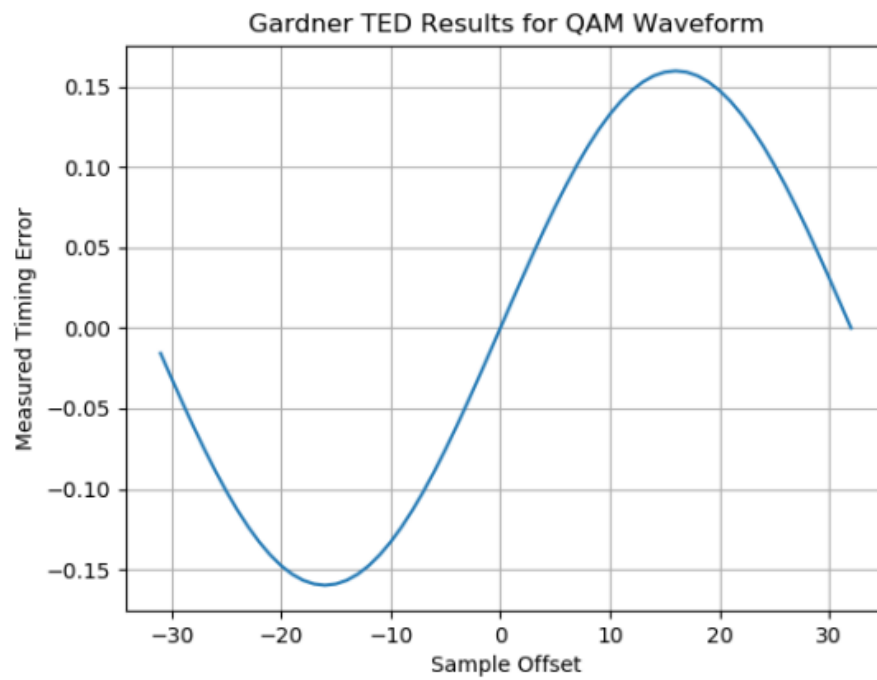


Рис. 33 — Форма TED

Фильтр контура

Фильтр выполнен в виде пропорционально-интегрирующего звена по схеме

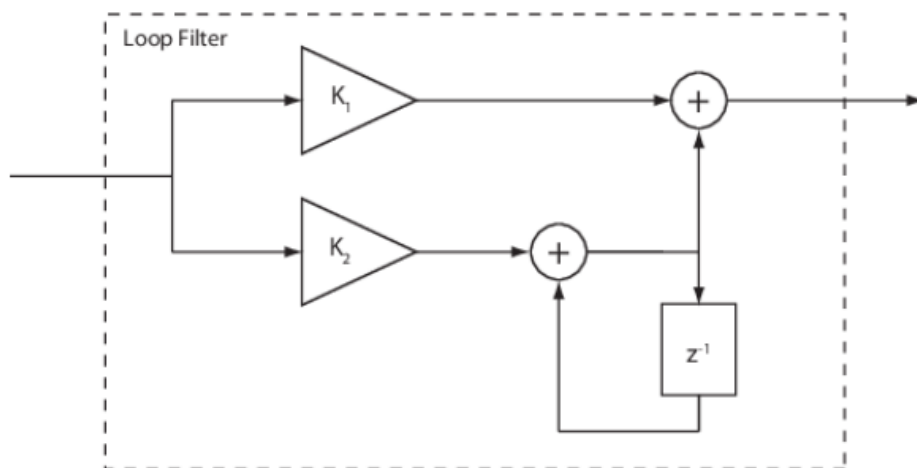


Рис. 34 — Форма TED

Параметры фильтра

$$K_1 = \frac{-4\zeta\theta}{(1 + 2\zeta\theta + \theta^2)K_p}$$

and

$$K_2 = \frac{-4\theta^2}{(1 + 2\zeta\theta + \theta^2)K_p}.$$

The interim term, θ , is given by

$$\theta = \frac{\frac{B_n T_s}{N_{sps}}}{\zeta + \frac{1}{4\zeta}},$$

where:

- N_{sps} is the number of samples per symbol.
- ζ is the damping factor.
- $B_n T_s$ is the loop bandwidth (B_n) normalized to the symbol rate (T_s).
- K_p is the detector gain.

Рис. 35 — Параметры фильтра

Программно фильтр реализуется в общем цикле по отсчетам сигнала на выходе СФ.

Начальные значения переменных в ветвях K1 и K2 - p1 и p2 равны 0. Сами величины K1 и K2 вычисляются перед циклом. Нормированная полоса фильтра контура $B_n T_s = 0.01$. Также перед циклом задается начальное смещение $offset = 0$ (номер отсчета символа, один из 10).

В цикле по отсчетам сигнала на выходе СФ вычисляется текущее значение ошибки e для текущего смещения $n = offset$;

$$e = (I(n+Nsp+Ns) - I(n+Ns)) * I(n+Nsp/2+Ns) + (Q(n+Nsp+Ns) - Q(n+Ns)) * Q(n+Nsp/2+Ns).$$

Если e - текущая величина ошибки на выходе детектора ошибки (на данном индексе отсчета сигнала на выходе СФ), то в ветви K1 переменная $p1 = e * K1$. В ветви K2 переменная $p2 = p2 + p1 + e * K2$. Эта переменная $p2$ имеет смысл задержки, дробного смещения ($offset$) на длительности символа. Тут же, если $p2 > 1$, ее сбрасывают

$$\text{while}(p2 > 1) p2 = p2 - 1; \text{end.}$$

После этого определяется уточненное смещение в отсчетах ($offset$), для этого нужно умножить $p2$ на $Nsps$ и округлить произведение до ближайшего целого, $offset = \text{round}(p2 * Nsps)$, это будет смещение на данной итерации (оптимальный отсчет текущего символа, индекс в массиве отсчетов сигнала на выходе СФ). На следующем символе снова вычисляется величина ошибки и после фильтрации определяется смещение, по которому находится следующий индекс отсчета $offset + Nsps$. Таким образом контур сводит ошибку к минимуму и определяются оптимальные индексы в массиве отсчетов символов сигнала на выходе СФ.

Выполнение

Напишем функцию Гарднера, которая будет автоматически искать offset

Листинг 7.1 — Функция Гарднера

```
1 import numpy as np
2 def gardner(complex_symbols_saw):
3     p1 = 0
4     p2 = 0
5     offset = 0
6     BnTs = 0.01
7     Nsp = 10 # Samples per symbol
8     zeta = np.sqrt(2)/2
9     teta = (BnTs/Nsp)/(zeta + 1/(4*zeta))
10    css = complex_symbols_saw
11    Kp = 0.002
12    K1 = (-4 * zeta * teta)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
13    K2 = (-4 * teta ** 2)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
14    list_of_offset = []
15
16    for n in range(0, len(css)-22):
17        err = (np.real(css[n + Nsp + Nsp]) - np.real(css[n + Nsp])) * np.real(css[n +
18            Nsp + Nsp//2]) + (np.imag(css[n + Nsp + Nsp]) - np.imag(css[n + Nsp])) *
19            np.imag(css[n + Nsp + Nsp//2])
20        err = np.real(err)
21        p1 = err * K1
22        p2 = p2 + p1 + err * K2
23        while p2 > 1:
24            p2 = p2 - 1
25        while p2 < -1:
26            p2 = p2 + 1
27        offset = np.round(p2 * Nsp)
28        list_of_offset.append(offset)
29
30    return int(offset), list_of_offset
```

Используем эту функций в нашем коде

Листинг 7.2 — Функция Гарднера

```
1 from matplotlib import pyplot as plt
2 import numpy as np
3 from auto_offset import gardner
4
5 rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
6     int16)
7
8 samples = []
```

```

8
9 for x in range(0, len(rx), 2):
10     samples.append((rx[x] + 1j * rx[x+1])/np.max(rx))
11
12 plt.figure()
13 plt.scatter(np.real(samples), np.imag(samples))
14 plt.axhline()
15 plt.axvline()
16
17 impulse = np.ones(10)
18
19 I = np.real(samples)
20 Q = np.imag(samples)
21
22 i_convolve = np.convolve(I, impulse)
23 q_convolve = np.convolve(Q, impulse)
24
25 i_con_dec = []
26 q_con_dec = []
27 list_offset = []
28
29 offset, list_offset = gardner(i_convolve + 1j * q_convolve)
30 print(offset)
31
32 plt.figure()
33 plt.title("offset")
34 plt.plot(np.arange(len(list_offset)), list_offset)
35 plt.ylim([-20, 20])
36 plt.xlim([4060, 4560])
37
38 for i in range(offset, len(i_convolve), 10):
39     i_con_dec.append(i_convolve[i])
40     q_con_dec.append(q_convolve[i])
41
42 # Убираем шумы
43 # i_con_dec = [x for x in i_con_dec if not (abs(x) <= 1)]
44 # q_con_dec = [x for x in q_con_dec if not (abs(x) <= 1)]
45
46 print("length i_con_dec: ", len(i_con_dec))
47 print("length q_con_dec: ", len(q_con_dec))
48
49 plt.figure()
50 plt.subplot(2,1,1)
51 plt.scatter(i_con_dec, q_con_dec)
52 plt.axhline()
53 plt.axvline()
54 plt.subplot(2,1,2)
55 plt.xlim([5000, 5500])
56 plt.ylim([-30, 30])

```

```
57 plt.scatter(np.arange(len(i_convolve)), i_convolve)
58 plt.scatter(np.arange(len(q_convolve)), q_convolve)
59
60 plt.show()
```

Вывод

Написал, программную реализация детектора ошибки, исполнил в коде схему Гарднера.